

J

JOAccountant — Backend v5.2

AUDIT MULTI-EXPERTS — 5 DIMENSIONS

Audit 360° du backend comptable *JOAccountant* pour le marché haïtien

Analyse approfondie par 5 expertises — comptable & fiscaliste, architecte logiciel, ingénieur sécurité, lead developer qualité, ingénieur performance — en vue d'une mise en production sur le marché cible Haïti.

Fiscalité DGI Haïti

Architecture Spring Boot

Sécurité OWASP

Scalabilité multi-tenant

DevOps & CI/CD

VERDICT : NO-GO CONDITIONNEL**Destinataire**

Tech Lead — Fondateur
Rapport interne confidentiel

Édition du 29 juillet 2026
16 sections — ~45 pages

Résumé exécutif & Verdict Go/No-Go

Verdict final

NO-GO CONDITIONNEL

Le backend **JOAccountant v5.2** présente une **architecture solide et bien documentée** (30 modules Gradle, 190 endpoints REST, 63 migrations Flyway, 310 tests, 6 référentiels comptables) qui démontre une réelle maîtrise du DDD stratégique et de l'ingénierie Spring Boot. **Cependant, le projet n'est pas prêt pour une mise en production sur le marché haïtien en l'état.**

Trois catégories de blocages critiques ont été identifiées : (1) **3 vulnérabilités de sécurité critiques** (bypass MFA, clé MFA sans fail-fast, RLS PostgreSQL inactif), (2) **une inadaptation fiscale majeure** au contexte haïtien (calendrier 100% français, TCA absente, taux IS hardcoded à 25% au lieu de 30%, mentions légales factures 100% CGI), (3) **des lacunes ops structurantes** (audit_log non immutable, pipeline CI/CD sans scans sécurité, pagination manquante sur endpoints à fort volume).

Le délai réaliste pour atteindre un GO conditionnel est estimé à **6-8 semaines** via un plan d'action priorisé sur 30/60/90 jours. Les fondations techniques permettent une remontée rapide vers un état production-ready.

Scores par dimension d'expertise

Le tableau ci-dessous synthétise les scores obtenus par le projet sur les six dimensions d'évaluation mobilisées pour cet audit. Chaque score est justifié en détail dans la section dédiée correspondante de ce rapport.

Dimension	Score	Verdict	Section
Fiscalité & comptable (Haïti)	3 / 10	Critique	§6
Architecture & ingénierie	7.5 / 10	Solide	§7
Sécurité applicative	5.9 / 10	Insuffisant	§8
DevOps, observabilité & ops	7.0 / 10	Correct	§9
Dette technique & qualité	6.5 / 10	Moyen	§10
Scalabilité & performance	6.5 / 10	Moyen	§11
Conformité légale Haïti	3.0 / 10	Critique	§12
Score global pondéré	5.6 / 10	NO-GO conditionnel	—

Top 5 des risques critiques (P0)

Les cinq risques suivants sont des **bloquants absolus** pour toute mise en production. Ils doivent être intégralement corrigés avant le lancement, sans exception possible. Aucun ne relève de l'amélioration continue : chacun expose le projet à un risque juridique, fiscal ou de sécurité immédiat.

[P0] Bypass authentication MFA

Le endpoint `/auth/login/mfa` valide le `mfaChallengeToken` via `JwtService.parseClaims()` qui ne vérifie pas la signature JWT. Un attaquant peut forger un JWT non signé avec `sub=<userId victime>` et brute-forcer le code TOTP (1M combinaisons, endpoint non rate-limité). **Impact : prise de compte complète d'un utilisateur MFA-activé.**

[P0] Clé de chiffrement MFA sans fail-fast

La propriété `app.mfa.encryption-key` a une valeur par défaut `dev-only-mfa-key-please-override-32-chars` committée dans Git, et **aucun contrôle de démarrage** ne vérifie qu'elle a été surchargée en production. Si la variable d'environnement `APP_MFA_ENCRYPTION_KEY` est absente, tous les secrets TOTP des utilisateurs sont chiffrés avec une clé publique — déchiffrables par tout attaquant obtenant un accès à la base de données.

[P0] Row Level Security PostgreSQL inactif

La migration V51 active RLS sur 6 tables critiques (`journal_line`, `journal_entry`, `sales_invoice`, `purchase_invoice`, `third_party`, `expense_report`) mais **aucun code Java n'exécute** `SET LOCAL app.current_tenant = ?` au début des transactions. La défense en profondeur multi-tenant au niveau base de données est donc inerte — toute la sécurité repose sur la discipline manuelle des développeurs dans les 50+ services qui appellent `findById()` sans filtre tenant.

[P0] Calendrier fiscal 100% français

Le service `TaxService.getDeclarationSchedule()` génère un calendrier déclaratif exclusivement français : TVA mensuelle au 19 du mois M+1, IS solde au 15 mai N+1, DES intra-UE au 10. **Aucune équivalence DGI haïtienne n'existe** (TVA/TCA/RS mensuelles au 15 M+1, acompte IS 1% mensuel, DCR/DCRf annuelles au 31 mars N+1). Un utilisateur suivant ce calendrier sera systématiquement en retard de déclaration → pénalités Code Fiscal art. 240 (10 à 50% du montant dû).

[P0] Audit trail non immutable

La table `audit_log` n'a ni trigger `BEFORE UPDATE/DELETE`, ni `REVOKE UPDATE, DELETE`. Un attaquant (ou un admin DB négligent) peut altérer ou supprimer l'audit forensique. Pour un SaaS comptable soumis au Code Fiscal Haïtien (10 ans de rétention obligatoire) et au Décret du 23 mai 2018 sur les données personnelles, c'est une **non-conformité légale majeure**.

Méthodologie d'audit multi-experts

Cet audit a été conduit en mode **revue statique de code** sur l'intégralité du dépôt JOAccountant Backend v5.2 (archive joaccountant_backend_v5.3.1-.zip), sans exécution dynamique des tests ni pentest en boîte noire. L'objectif était de fournir une analyse 360° du projet sous cinq angles d'expertise complémentaires, afin d'évaluer sa maturité pour une mise en production sur le marché haïtien, qui constitue le public cible principal selon la volonté exprimée par le fondateur.

Les cinq expertises mobilisées

Expertise	Périmètre analysé	Livrable
Comptable & fiscaliste <i>Spécialiste DGI Haïti + CGI/SYSCOHADA/IFRS</i>	Modules tax, accounting-engine, financial-statements, payroll, invoicing, employees, funds-grants, fx-operations, fixed-assets, chart-of-accounts, bank-reconciliation. Toutes les migrations Flyway. Conformité DGI, TCA, IS, retenues à la source, CNSS, OFATMA, AST, DCR/DCRf, Factur-X, Code Fiscal art. 196.	Score maturité Haïti par module, top 5 risques fiscaux, top 10 actions prioritaires.
Architecte logiciel <i>DDD, Spring Boot multi-module, multi-tenant SaaS</i>	settings.gradle.kts, build.gradle.kts (root + 30 modules), core/ (socle), structure de chaque module, inter-dépendances, TenantContextFilter, RLS PostgreSQL, ArchUnit, app/, deploy/, Dockerfile, .github/workflows.	Score par dimension architecture, top 5 risques, top 5 forces, top 10 actions.
Ingénieur sécurité <i>OWASP Top 10, NIST 800-63B, ISO 27001, DevSecOps</i>	auth/ (JWT HS256/RS256, MFA TOTP, Argon2id, RefreshToken, JWKS), audit-trail/, core/security/, application.yml, Dockerfile, .github/workflows, OPERATIONS.md, Helm chart. Couverture OWASP Top 10 complète.	Score par dimension sécurité, top 5 vulnérabilités, top 5 forces, top 10 actions P0/P1/P2.
Lead developer qualité <i>Dette technique, refactoring, standards Java/Spring</i>	464 fichiers Java (main) + 31 fichiers de test. Volumétrie, god classes, couverture tests, conventions, code smells, dépendances externes (libs.versions.toml), configuration, documentation, build.	Score par dimension qualité, top 5 zones de dette, top 5 forces, top 10 actions.
Ingénieur performance <i>PostgreSQL, indexing, query optimization, scale</i>	Toutes les migrations (CREATE INDEX, EXPLAIN, partitioning), requêtes JPA, N+1, multi-tenant scale, pagination, batch, cache, connection pooling, tables à fort volume, tests de charge Gatling.	Score par dimension perf, top 5 goulots d'étranglement, top 5 forces, top 10 actions.

Périmètre analytique

Le périmètre couvre l'intégralité du code source du backend : **30 modules Gradle** (27 modules métier + 1 module utilitaire de test + 1 module d'application + 1 module purchase-orders), **464 fichiers Java** principaux pour ~46 000 lignes de code, **31 fichiers de test** pour ~10 000 lignes, **63 migrations Flyway** pour ~3 500 lignes SQL, **3 scripts de seed** Python (commerce retail, wholesale, services professionnels), **~190 endpoints REST** distribués sur **32 contrôleurs**, et la documentation (README.md 720 lignes, 28 README par module, OPERATIONS.md 317 lignes).

L'audit ne couvre pas : (1) l'**application mobile** livrée (APK 21.7 MB, 16 feature modules) qui n'était pas dans l'archive — son évaluation sécurité et UX est recommandée en complément ; (2) les **tests dynamiques** (exécution de la suite de tests, tests de charge réels, pentest) ; (3) la **conformité contractuelle** avec les CGU/CGV SaaS, qui relève du juridique ; (4) l'**expérience utilisateur** côté frontend web, non livré dans cette archive.

Critères d'évaluation

Chaque dimension est notée sur 10 selon une grille explicite : **0-3** = critique, non production-ready avec risques juridiques/sécurité majeurs ; **4-5** = insuffisant, des correctifs P0/P1 nécessaires avant production ; **6-7** = correct, des améliorations P1/P2 souhaitables ; **8-9** = solide, prêt pour production avec surveillance continue ; **10** = exemplaire, référence sectorielle. Le score global pondéré accorde une importance double à la sécurité et à la fiscalité Haïti (poids 2), un poids normal à l'architecture, la dette technique et la scalabilité (poids 1), et un poids 0.5 à la conformité légale et au DevOps (qui sont des dimensions complémentaires).

Vue d'ensemble du projet

JOAccountant Backend v5.2 est un SaaS comptable multi-tenant, multi-secteur et multi-référentiel, bâti sur Spring Boot 3.5 / Java 17 / PostgreSQL 14+ / Flyway. Le projet est structuré en **30 modules Gradle** organisés selon les principes du Domain-Driven Design stratégique : chaque module représente un bounded context métier avec son propre domaine, ses entités, ses services, ses contrôleurs REST et ses migrations de base de données. L'ambition affichée dans le README est de couvrir l'intégralité du cycle comptable d'exploitation, de la saisie d'écritures à la génération des états financiers, en passant par la facturation, la paie, les immobilisations, la trésorerie et la fiscalité.

Caractéristiques techniques clés

Caractéristique	Valeur
Stack principal	Spring Boot 3.5.0, Java 17, PostgreSQL 14+, Flyway Community, Hibernate 6.5
Modules Gradle	30 (27 métier + test-support + app + purchase-orders)
Endpoints REST	~190 sur 32 contrôleurs, préfixe /api/v1
Migrations DB	63 (V0_000 → V52), ~3 500 lignes SQL
Tests	310 tests d'intégration sur 31 fichiers + 42 règles ArchUnit
Référentiels comptables	6 : IFRS_FULL, IFRS_SME, SYSCOHADA_REVISED, PCG_FRANCE, PCN_HAITI, PCGR_CANADA
Types métier	13 (retail, wholesale, services, ONG, etc.) extensibles sans redéploiement
Multi-tenant	Shared DB + shared schema + tenant_id (company_id) discriminator
Sécurité	JWT HS256/RS256, MFA TOTP RFC 6238, Argon2id, JWKS RFC 7517, RLS PostgreSQL V51
Observabilité	Micrometer + Prometheus + OpenTelemetry + Logback JSON (MDC correlationId/userId/companyId)
DevOps	Dockerfile multi-stage JRE 21, Helm chart (HPA 2 → 10, probes, security context), GitHub Actions CI
Backup	pg_basebackup + WAL archiving (PITR), RPO < 5 min, RTO < 1h, test mensuel automatisé, rétention 10 ans S3 Glacier

Cartographie des 30 modules

Le tableau ci-dessous présente les 30 modules Gradle avec leur rôle et leur catégorie. Cette cartographie servira de référence tout au long du rapport, chaque section d'analyse faisant référence aux modules concernés.

Module	Rôle	Catégorie
core	Socle : TenantAwareEntity, TenantContext, exceptions, ProblemDetail RFC 7807, ports hexagonaux, AccountingFramework	Infrastructure
audit-trail	AuditLog + AuditEventListener async AFTER_COMMIT — seule entité NON-TenantAware	Infrastructure
auth	User, RefreshToken rotation, MFA TOTP, JWKS, JWT HS256/RS256, Argon2id, password validator OWASP	Domaine

Module	Rôle	Catégorie
company	Company (=tenant), CompanyModule, BusinessType, wizard 9 étapes, ModuleAccessGuard, MaxCompaniesGuard	Domaine
document-numbering	DocumentSequenceConfig + Counter, upsert atomique ON CONFLICT DO UPDATE, numérotation sans trou	Domaine
chart-of-accounts	Account (hiérarchie 4 niveaux), AccountNumberingTemplate, inferReportingClass multi-référentiel, SectorAccountTemplate	Domaine
approval-workflow	ApprovalRule + ApprovalRequest, quatre yeux maker-checker, multi-étapes	Domaine
analytics	AnalyticalDimensionPlan + Value, générique multi-secteur	Domaine
accounting-engine	JournalEntry (DRAFT/PENDING/POSTED/VOIDED), JournalLine, FiscalYear/Period, trigger DB équilibre, clôture	Domaine cœur
financial-statements	FinancialStatementSnapshot, bilan, CR, cash flow IAS 7, snapshots de clôture idempotents	Reporting
third-parties	ThirdParty, LettrageMatch, balance âgée clients/fournisseurs	Domaine
fixed-assets	Asset, AssetComponent (IAS 16), DepreciationScheduleLine, ImpairmentTest (IAS 36)	Domaine
inventory	Item, StockMove, Warehouse, StockValuationLayer, FIFO/AVG, COGS auto	Sectoriel
time-billing	Project, TimesheetEntry, BillableRate, WIP (secteur SERVICE)	Sectoriel
document-generation	PDF via Thymeleaf + openhtmltopdf (bilan, CR, facture, bulletin de paie)	Infrastructure
invoicing	SalesInvoice, InvoiceLine, Factur-X (CII D16B BASICWL EN 16931), reverse charge	Domaine
bank-reconciliation	BankStatement, BankStatementLine, auto/manual match, parsers CSV/OFX/MT940	Sectoriel
funds-grants	Grant, DonationReceipt, ExpenseTaggedToGrant (secteur ONG)	Sectoriel
notifications	Notification, NotificationTemplate, AlertRule, listeners forensiques	Infrastructure
tax	TaxRule, WithholdingRule (barème progressif V46), CorporateTaxRule, ContributionRule, VatMode	Domaine
reporting	Orchestration des 15 exports PDF/CSV, Dashboard, balances âgées (dépend de 19 modules)	Reporting
purchasing	PurchaseInvoice, InvoiceLine, écriture Débit Achats+TVA déductible/Crédit Fournisseur	Domaine
purchase-orders	PurchaseOrder, ThreeWayMatchService (commande ↔ facture)	Domaine
expenses	ExpenseReport, ExpenseLine, ExpenseCategory (plafonds V43)	Domaine
employees	Employee (rattaché ThirdParty EMPLOYEE), aucune écriture	Domaine
payroll	PayrollRun, Payslip, PayrollCalculator via WithholdingRule+ContributionRule, Spring Batch payrollJob	Domaine
fx-operations	FxOperation (BUY/SELL/REVALUATION), ExchangeRate, gain/perte de change	Sectoriel
test-support	EmbeddedPostgresSupport (Zonky) — module utilitaire	Test
app	@SpringBootApplication, SecurityConfig, BatchConfig, OpenApiConfig, tests intégration	Composition

L'architecture modulaire est l'un des points forts du projet : chaque module possède son propre `build.gradle.kts`, son propre README, et ses propres migrations Flyway. L'isolation compile-time

est garantie par 42 règles ArchUnit qui vérifient notamment l'absence de dépendances circulaires, la pureté du socle :core (aucune dépendance sortante), et le respect des conventions de nommage des repositories d'entités tenant-aware. Cette discipline d'ingénierie est rare pour un projet de cette envergure.

Cartographie des scores par dimension

Le graphique radar ci-dessous synthétise les scores obtenus par JOAccountant Backend v5.2 sur les sept dimensions d'évaluation. La forme générale révèle un projet **solide sur l'architecture et le DevOps/observabilité**, mais **en retrait marqué sur la fiscalité Haïti, la sécurité applicative et la conformité légale**. Cette dissymétrie est révélatrice : le projet a été bâti avec un référentiel français (CGI, SYSCOHADA, Loi 2023-314 Factur-X) en tête, sans adaptation spécifique au contexte haïtien malgré la présence nominale du référentiel PCN_HAITI.

Détail des scores par sous-dimension

Dimension	Sous-dimension	Score	Statut
Fiscalité Haïti	Référentiels comptables (PCN_HAITI)	4/10	Critique
Fiscalité Haïti	TVA / TCA	3/10	Critique
Fiscalité Haïti	Impôt sur Société	2/10	Critique
Fiscalité Haïti	Retenues à la source / cotisations	5/10	Moyen
Fiscalité Haïti	Paie (Code du Travail Haïti)	2/10	Critique
Fiscalité Haïti	États financiers / DCR DGI	3/10	Critique
Fiscalité Haïti	Facturation électronique	3/10	Critique
Fiscalité Haïti	Déclarations DGI	0/10	Absent
Fiscalité Haïti	Banque / rapprochement	6/10	Correct
Architecture	Organisation modules	8/10	Solide
Architecture	Multi-tenant	6/10	Moyen
Architecture	DDD tactique	5/10	Moyen
Architecture	Dépendances (ArchUnit)	8/10	Solide
Architecture	API design (RFC 7807)	8/10	Solide
Architecture	Base de données / migrations	8/10	Solide
Sécurité	Authentification	6.5/10	Moyen
Sécurité	Autorisation / multi-tenant	6/10	Moyen
Sécurité	Gestion secrets	5/10	Insuffisant
Sécurité	Audit trail immutabilité	6/10	Moyen
Sécurité	OWASP Top 10	6.5/10	Moyen
DevOps	CI/CD pipeline	3.5/10	Critique
DevOps	Observabilité	7/10	Correct
DevOps	Backup & DR	7.5/10	Solide
DevOps	Helm chart K8s	7/10	Correct
Qualité	Couverture tests unitaires	3/10	Critique
Qualité	Conventions code	8/10	Solide
Qualité	Documentation	9/10	Exemplaire

Dimension	Sous-dimension	Score	Statut
Qualité	Build & lint	6/10	Moyen
Scalabilité	Indexation PostgreSQL	8/10	Solide
Scalabilité	Pagination	6/10	Moyen
Scalabilité	Requêtes JPA / N+1	5/10	Insuffisant
Scalabilité	Multi-tenant scale	6/10	Moyen
Scalabilité	Connection pooling	7/10	Correct
Conformité légale	Décret 23 mai 2018 (données perso)	3/10	Critique
Conformité légale	Conservation pièces comptables	5/10	Insuffisant
Conformité légale	Signature électronique (Décret 2002)	2/10	Critique

Lecture du score global

Le **score global pondéré de 5.6/10** traduit un projet techniquement mature sur l'architecture et l'ingénierie, mais dont la trajectoire de développement s'est éloignée du marché cible haïtien. La pondération double accordée à la sécurité et à la fiscalité Haïti fait chuter le score global malgré l'excellence sur les dimensions DevOps et scalabilité : un SaaS comptable qui se trompe de référentiel fiscal ou qui présente une vulnérabilité critique d'authentification ne peut pas être déployé en production, quand bien même son architecture serait exemplaire.

Lecture stratégique

Le projet est **engagé sur la bonne voie architecturale** : découpage DDD stratégique exemplaire, observabilité production-grade, backup PITR testé mensuellement, stack moderne. Ces fondations permettront une remontée rapide vers l'état production-ready.

Mais la trajectoire fiscale doit être corrigée : le projet est actuellement calibré pour le marché français (CGI, SYSCOHADA, Factur-X Loi 2023-314) alors que le public cible est haïtien. Sans cette correction, le projet ne peut pas être déployé légalement en Haïti.

Et 3 vulnérabilités critiques de sécurité doivent être corrigées avant tout go-live, indépendamment du contexte fiscal : bypass MFA, clé MFA sans fail-fast, RLS inactif.

Analyse fiscale & comptable pour Haïti

Cette section constitue le cœur de l'audit pour le marché haïtien. Le constat central est le suivant : **le projet JOAccountant a été bâti avec un référentiel fiscal français en tête** (CGI, SYSCOHADA révisé, Loi 2023-314 sur la facturation électronique, Factur-X EN 16931), alors que le fondateur déclare que le public cible est haïtien. Le référentiel `PCN_HAITI` existe bien dans l'énumération `AccountingFramework` (migration `V1_002`), mais l'analyse du code révèle qu'il s'agit d'une **coquille vide** : aucune spécialisation n'est implémentée pour ce référentiel, qui se comporte en pratique comme un SYSCOHADA renommé.

L'impact concret est majeur : une entreprise haïtienne qui utiliserait JOAccountant en l'état produirait des déclarations fiscales incorrectes (taux d'IS à 25% au lieu de 30%, calendrier déclaratif inadapté, absence de TCA), des factures non conformes au Code Fiscal haïtien (mentions légales françaises), et des bulletins de paie non conformes au Code du Travail haïtien (heures supplémentaires calculées sur 35h/semaine au lieu de 48h). Chacune de ces non-conformités expose l'entreprise utilisatrice à des redressements fiscaux et à des pénalités substantielles.

Score global fiscalité Haïti : 3 / 10 — Critique

Le projet n'est pas déployable légalement en Haïti en l'état. Les écarts fiscaux identifiés exposent les utilisateurs à des redressements DGI systématiques. Un investissement estimé à **~49 jours-homme** est nécessaire pour atteindre une conformité fiscale haïtienne acceptable (avec un expert-comptable local DGI en relecture).

6.1 Référentiels comptables supportés

Le seed `V1_002_core_seeds.sql` initialise six référentiels comptables dans la table `accounting_framework` : `IFRS_FULL`, `IFRS_SME`, `SYSCOHADA_REVISED`, `PCG_FRANCE`, `PCN_HAITI` et `PCGR_CANADA`. Chacun déclare son `accountNumberingType` (`FREE` pour IFRS, `MANDATED` pour les autres), le nombre de classes comptables (7 pour PCG France, 8 pour SYSCOHADA et PCN Haïti, 6 pour IFRS), et la liste des états financiers obligatoires.

L'analyse du service `ChartOfAccountsService.inferReportingClass()` (lignes 596-676) révèle que seule la branche `PCGR_CANADA` dispose d'une logique de mapping spécifique. Le référentiel `PCN_HAITI` suit le **même mapping que SYSCOHADA et PCG France** (classe 8 traitée comme HAO, deuxième chiffre utilisé pour distinguer charges/produits). Or, le Plan Comptable National haïtien utilise la classe 8 pour les « Comptes spéciaux » (engagements hors bilan, comptes de régularisation), et non pour les HAO comme en SYSCOHADA. Cette incohérence produit des états financiers incorrects pour toute société haïtienne utilisant des comptes de classe 8.

Plus préoccupant encore : la migration `V50__vat_accounts_447_4438_seeds.sql` (lignes 60-61) seede les comptes 447 (« État — TVA à reverser ») et 4438 (« TVA différée ») **uniquement pour SYSCOHADA_REVISED et PCG_FRANCE**. Pour `PCN_HAITI`, ces comptes ne sont jamais créés. Conséquence pratique : la première facture encaissée en TVA différée ou en autoliquidation lèvera une exception `VAT_DEFERRED_ACCOUNT_NOT_FOUND`, empêchant la facturation électronique de fonctionner pour une société haïtienne.

Recommandations prioritaires

- **[P1]** Créer un `PcnHaitiAccountTemplate.java` (sibling de `SectorAccountTemplate`) qui ajoute les classes 8 « Spéciaux » et 9 PCN haïtien, plus les comptes 4456/4457 (TVA déductible/collectée

PCN), 442 (« État — retenues à la source »), 446 (« État — TCA »), 447 (« État — IS »), 448 (« État — taxes diverses »).

- [P1] Étendre la migration V50 pour seed les comptes 447, 4438, 442, 446 également lorsque `framework.code = 'PCN_HAITI'`.
- [P2] Ajouter une branche `inferReportingClass` spécifique à `PCN_HAITI` (classe 8 haïtienne ≠ HAO SYSCOHADA).
- [P3] Tester `PCN_HAITI` end-to-end (couverture actuelle 1 test sur 18 → cible $\geq 5/18$) avec un seed `seed_ong_haiti.py` couvrant le cas d'usage ONG.

6.2 Fiscalité — TVA et TCA

L'entité `TaxRule` (module `tax`) gère les taux de TVA de façon configurable entreprise par entreprise. Le champ `rate` est un `BigDecimal(5,2)` dont le commentaire (ligne 45) mentionne explicitement « ex. 15.00 pour 15% TVA Haïti » — ce qui démontre que l'intention haïtienne était présente dès la conception. Le script `seed_commerce.py` (ligne 412) crée effectivement une `TaxRule` avec un taux de 10% (TVA haïtienne, loi du 24 juillet 2002), ce qui est correct.

Cependant, plusieurs limites critiques subsistent. Premièrement, **aucune TCA (Taxe sur le Chiffre d'Affaires)** n'est modélisée. Or la TCA haïtienne est une taxe distincte de la TVA, perçue sur certains services (Code Fiscal art. 196) : 2% sur les opérations bancaires, 5% sur les télécommunications, 10% sur les autres services. L'entité `TaxRule` ne dispose pas de champ `taxType` pour distinguer TVA vs TCA vs IS vs retenue — c'est un manque structurel qui empêche de gérer correctement la fiscalité haïtienne.

Deuxièmement, le mode `VatMode.ENCAISSEMENT` (migration V44) implémente la TVA sur encaissement de l'article 289 II du CGI français (compte 4438 « TVA différée »). Or en Haïti, la TVA est exigible dès la facturation (Code Fiscal art. 191), donc le mode `DEBIT` par défaut convient. Mais l'option `ENCAISSEMENT` reste exposée dans l'interface, ce qui peut induire les utilisateurs en erreur. Troisièmement, le mécanisme de reverse-charge (V45, `sales_invoice.is_reverse_charge`) est codé en dur pour l'autoliquidation intra-UE B2B (article 283 2 nonies CGI) — inutilisable en Haïti.

Enfin, le calcul de la TVA due (TVA collectée – TVA déductible) est correct dans `TaxService.getDeclaration()` (lignes 242-257), mais le crédit de TVA résultant (quand la déductible dépasse la collectée) **n'est pas persisté** (commentaire TODO ligne 247). La déclaration suivante repart de zéro, ce qui fait perdre à l'entreprise son crédit de TVA — une perte de trésorerie directe.

Recommandations prioritaires

- [P1] Ajouter un champ `TaxRule.taxType` (enum `VAT, TCA, TURNOVER_TAX, EXCISE`) pour distinguer les taxes haïtiennes. Migration V53.
- [P1] Seeder des `TaxRule` globales (`companyId IS NULL`) pour Haïti : `TVA_HT_10` (10%), `TCA_HT_2_BANK` (2% sur opérations bancaires, art. 197), `TCA_HT_5_TELECOM` (5% télécoms), `TCA_HT_10_SERVICES` (10% autres services).
- [P2] Implémenter la persistance du crédit de TVA (TODO `TaxService.java:247`) via une table `tax_credit_carried_forward`.
- [P2] Documenter `VatMode.ENCAISSEMENT` comme non-applicable en Haïti (la TVA y est toujours sur débits) — ou filtrer cette option si `country='HT'`.
- [P3] Généraliser le reverse-charge à d'autres cas (prestataires non-résidents en Haïti, art. 197 bis du Code Fiscal).

6.3 Fiscalité — Impôt sur Société (IS)

L'Impôt sur Société est géré par l'entité `CorporateTaxRule` et le service `TaxService.projectCorporateTax()` (lignes 302-388). L'architecture est saine : taux standard, taux réduit PME, seuil d'éligibilité, calcul des acomptes trimestriels et du solde annuel. Le problème est que **toutes les valeurs par défaut sont françaises**.

Plus précisément, la méthode `defaultFrenchCorporateTaxRule()` (lignes 391-398) constitue un fallback hardcodé : `standardRate=25`, `reducedRate=15`, `reducedRateThreshold=42500`. Si une entreprise ne configure pas de `CorporateTaxRule`, c'est le **barème français 2026 qui s'applique par défaut**, y compris pour une société haïtienne. Or l'IS haïtien est très différent : taux normal de 30% sur le bénéfice net (Code Fiscal art. 4), taux réduit de 15% pour les entreprises en zones franches, acompte de 1% sur les encaissements (art. 5 du Code Fiscal), solde annuel au 31 mars N+1 (et non 15 mai N+1 comme en France).

Le calendrier déclaratif de l'IS est également exclusivement français : `TaxService.getDeclarationSchedule()` (lignes 426-506) génère les acomptes aux 15 mars, 15 juin, 15 septembre et 15 décembre (article 1668 CGI) et le solde au 15 mai N+1. Pour Haïti, le calendrier DGI est radicalement différent : acompte mensuel de 1% sur encaissements (15 du mois M+1) et déclaration annuelle DCR au 31 mars N+1. L'écart de 6 semaines entre le 15 mai et le 31 mars expose l'utilisateur à des pénalités de retard systématiques.

Risque fiscal critique — IS calculé au taux français par défaut

Si une entreprise haïtienne ne configure pas explicitement de `CorporateTaxRule`, son IS est calculé à 25% au lieu de 30%. En cas de contrôle DGI : redressement systématique (5 points d'écart × résultat fiscal = plusieurs millions de HTG) + pénalités 50% + intérêts de retard 1%/mois. Ce risque concerne potentiellement toutes les PME haïtiennes qui utiliseraient le SaaS sans configuration fiscale préalable.

Recommandations prioritaires

- [P0] Remplacer `defaultFrenchCorporateTaxRule` par un `defaultCorporateTaxRule(country)` qui renvoie IS=30% (HT), IS=25% (FR), IS=15%/25% selon éligibilité (FR PME), etc.
- [P0] Ajouter un `CorporateTaxRegime` enum (FR_NORMAL, FR_SME, HT_NORMAL, HT_FREE_ZONE, HT_NON_PROFIT, HT_SMALL_BUSINESS) pour distinguer les barèmes.
- [P1] Calendrier déclaratif IS contextuel (15 mai N+1 pour FR ; 31 mars N+1 pour HT — DCR annuelle).
- [P1] Modéliser l'acompte IS haïtien 1% sur encaissements (art. 5 du Code Fiscal) — distinct des acomptes trimestriels français.
- [P2] Implémenter la retenue à la source sur ventes (RAS 1%) haïtienne (art. 156) au niveau de `InvoicingService.generateInvoiceEntry`.

6.4 Retenues à la source et cotisations sociales

L'architecture des retenues à la source est l'un des points forts du projet côté fiscalité. L'entité `WithholdingRule` supporte deux modes de calcul : FLAT (taux unique) et PROGRESSIVE (barème par tranches, migration V46). La javadoc mentionne explicitement « Pertinent pour la retenue à la source sur prestations de service, fréquente en Haïti et en zone OHADA » — ce qui démontre là encore une intention haïtienne originelle. Le service `PurchasingService.calculateSupplierWithholding()` (lignes 397-447) calcule correctement la RS sur la base HT (conforme à la pratique OHADA et haïtienne — Code Fiscal art. 156), crédite le compte 442000 (« État — retenues à la source »).

Le contrôleur `TaxController.java` (lignes 140-150) expose même un exemple Swagger explicite `RHS_HT_2` « Retenue à la source 2% Haïti (prestations) » applicable à `["SUPPLIER"]`. C'est la seule référence explicite à Haïti dans toute l'API fiscale.

Cependant, **aucune règle haïtienne n'est seedée par défaut**. L'utilisateur doit tout configurer manuellement, ce qui est un frein majeur à l'adoption. De même, l'entité `ContributionRule` (cotisations sociales) dispose d'une énumération `regime` incluant `HT_GENERAL` (régime haïtien — OFATMA + IR), mais ce régime est **uniquement mentionné dans l'énumération** : aucun seed, aucun calcul spécifique OFATMA, aucun exemple Swagger `HT_GENERAL`. La CNSS haïtienne (6% employeur + 6% salarié, plafonnée à 6 PMT), l'OFATMA Santé (3% + 1%), l'OFATMA Accidents du travail (2-3% selon secteur) et l'AST (Ajustement Social Temporaire, barème progressif par tranches mensuelles) sont totalement absents.

Recommandations prioritaires

- **[P0]** Seeder des `WithholdingRule` globales HT : RS 2% sur prestations locales (art. 156), RS 10% sur royalties, RS 30% sur services non-résidents, RAS 1% sur ventes gouvernementales.
- **[P0]** Seeder des `ContributionRule` `HT_GENERAL` : CNSS 6% employeur + 6% salarié (PMT = $6 \times \text{SMG} \approx 150\,000 \text{ HTG/mois}$), OFATMA Santé 3% + 1%, OFATMA Accidents 2-3%, AST barème progressif 2024 par tranches mensuelles.
- **[P1]** Ajouter `ContributionBase.PMT_CAPPED` (plafond CNSS spécifique, distinct du PMSS français).
- **[P2]** Créer un `HaitianPayrollRegimeSeeder.java` qui peuple ces règles automatiquement à l'initialisation d'une entreprise `country='HT'`.

6.5 Paie — Conformité au Code du Travail haïtien

Le module `payroll` et la classe `PayrollCalculator` présentent le même défaut que le reste du projet : **des constantes hardcodées françaises**. La durée légale mensuelle est fixée à `MONTHLY_LEGAL_HOURS = 173.33` (lignes 87), ce qui correspond à la durée légale française ($35\text{h/semaine} \times 52/12$). En Haïti, la durée légale est de $48\text{h/semaine} \times 52/12 \approx 208\text{h/mois}$ (Code du Travail art. 32). Conséquence : le taux horaire calculé pour un salarié haïtien sera surévalué de ~17%, faussant tous les calculs de paie en aval.

Les heures supplémentaires (`OVERTIME_25_RATE = 1.25`, `OVERTIME_50_RATE = 1.50`) correspondent au Code du Travail français (35h/sem, majoration 25% sur 4 premières HS, 50% au-delà). En Haïti (Code du Travail art. 36 et Loi 2017), c'est **+50% sur les heures 49-56** et **+100% au-delà de 56h ou les dimanches et jours fériés**. Le modèle `overtimeHours25/50` ne permet pas de représenter correctement les heures sup haïtiennes. Il faudrait ajouter `overtimeHours100`.

Le calcul des absences utilise un prorata sur 30 jours/mois (lignes 81), alors qu'en Haïti le congé annuel est de 15 jours ouvrables (Code du Travail art. 156) et le prorata devrait être sur 26 jours ouvrables/mois. L'énumération `ContractType` (`PERMANENT`, `FIXED_TERM`, `CONSULTANT`) ne couvre pas l'apprentissage (Code du Travail haïtien art. 4), le travail saisonnier agricole, ni le temps partiel.

Plus grave encore, le template de bulletin de paie (migration `V39__legal_mentions_invoice_credit_payslip.sql`, lignes 209-313) mentionne **exclusivement des références françaises** : SIRET, URSSAF, Code APE, « C. trav. R3243-1 ». Aucune mention haïtienne (CNSS, OFATMA, AST, matricule employé, Code du Travail haïtien art. 153). Sanction Code du Travail art. 156 : amende 1 000 à 5 000 HTG par bulletin non conforme + dommages-intérêts salarié.

Recommandations prioritaires

- [P0] Rendre `MONTHLY_LEGAL_HOURS` configurable par entreprise (champ `Company.monthlyLegalHours` ou dérivé du `country` : 173.33 pour FR, 208 pour HT).
- [P0] Étendre `Employee` avec `overtimeHours100` ET `overtimeHours50` (pas seulement 25/50) ; ajouter `cnssNumber`, `ofatmaSectorCode`.
- [P0] Créer un template bulletin de paie haïtien (CGI haïtien + Code du Travail haïtien) avec mentions légales CNSS/OFATMA.
- [P1] Ajouter `ContractType.APPRENTICE` et `SEASONAL` et le régime apprentissage Code du Travail haïtien art. 4.
- [P2] Modéliser le 13^e mois (obligatoire en Haïti, Code du Travail art. 153 — le « mois bonus » de fin d'année).

6.6 États financiers et exportation DGI

Le module `financial-statements` est l'un des plus aboutis du projet. Le service `FinancialStatementsService` calcule le bilan, le compte de résultat et le tableau de flux de trésorerie (Cash Flow Statement IAS 7 / SYSCOHADA TAFIRE) via l'énumération `ReportingClass`, ce qui rend le calcul agnostique du référentiel comptable. C'est la conception la plus mature du projet, à conserver.

Cependant, deux problèmes persistent. Premièrement, le **TAFIRE n'est pas implémenté** même pour SYSCOHADA : il est listé comme obligatoire dans `V1_002` ligne 44 (`mandatory_statements` pour SYSCOHADA), mais aucun type `FinancialStatementType.TAFIRE` n'existe dans l'énumération. Le tableau de flux est calculé mais non persisté sous ce type. Pour `PCN_HAITI`, `mandatory_statements` se limite à `BALANCE_SHEET`, `INCOME_STATEMENT` — pas de TAFIRE, pas de notes annexes.

Deuxièmement, **aucune exportation conforme DGI Haïti n'existe**. Pas de génération de DCR (Déclaration Comptable et Fiscale Résumée), DCRf (feuillet 1 résumé), DCRG (grand livre), DCLS (états financiers). Le module `reporting` expose 15 exports CSV/PDF mais aucun au format DGI. Le service `TaxExportService` exporte uniquement au format CA3 français (CSV avec BOM UTF-8, format `base_ht;taux_tva;tva_collectee;tva_deductible;tva_due`). Les formats DES et EFI renvoient 501 Not Implemented.

Recommandations prioritaires

- [P0] Implémenter un `DgiHaitiExportService` avec endpoints `/reporting/dgi/dcr`, `/reporting/dgi/dcrf`, `/reporting/dgi/dcrg`, `/reporting/dgi/dcls` au format attendu par la DGI (souvent Excel/CSV structuré avec en-têtes DGI normalisés).
- [P1] Ajouter `FinancialStatementType.TAFIRE` et l'implémenter pour SYSCOHADA (déjà listé comme obligatoire dans `V1_002`).
- [P1] Pour `PCN_HAITI`, ajouter `mandatory_statements = 'BALANCE_SHEET, INCOME_STATEMENT, NOTES_ANNEXES, DCR'` (la DCR est l'équivalent TAFIRE haïtien).
- [P2] Étendre le cash flow avec la ventilation « activités d'exploitation / investissement / financement » au format DGI Haïti.

6.7 Facturation électronique

Le module `invoicing` implémente Factur-X (Cross Industry Invoice D16B, profil BASICWL, EN 16931) via `FacturXExporter.java`. Cette implémentation est **exclusivement conforme à la Loi 2023-314 française** sur la facturation électronique B2B. Le profil utilisé est `urn:cen.eu:en16931:2017#compliant#urn:fnfe-mpe.org:factur-x:1.0.07:basicwl`, totalement

étranger au contexte haïtien.

Plus précisément, la méthode `writeTradeParty()` (lignes 233-246) sérialise l'identifiant SIRET et le numéro de TVA intracommunautaire, mais **pas le NIF haïtien**. Or le NIF de l'entreprise est bien stocké en base (champ `Company.nif`, migration V42), il n'est simplement pas sérialisé dans le XML Factur-X. Pour la DGI haïtienne, le NIF est obligatoire sur les factures depuis l'arrêté du 4 octobre 2017.

La méthode `embedFacturXInPdf()` (lignes 145-158) **lève `UnsupportedOperationException`** : la dépendance `openpdf/iText` n'est pas ajoutée au classpath du module invoicing. En conséquence, le PDF/A-3 conforme Factur-X n'est pas disponible — seul le XML est exposé via `/invoices/{id}/factur-x`. C'est un bloquant pour toute facturation électronique conforme.

Les mentions légales des factures (migration V39 lignes 113-127) sont en dur et 100% françaises : « Mentions legales (CGI art. 289 + C. com. art. L441-10) », « Penalites de retard: 3x taux d'interet legal », « Indemnité forfaitaire pour frais de recouvrement: 40 EUR ». **Aucune mention du Code Fiscal haïtien** (art. 196 — mentions obligatoires : NIF émetteur, raison sociale, adresse, date, désignation, montant HT, taux TVA/TCA, montant taxes). Sanction : amende 5 000 à 50 000 HTG par facture non conforme. Les champs SIRET/VAT/NIF de l'entité `Company` (V42) sont validés avec des patterns français (SIRET 14 chiffres, VAT format UE), non haïtiens (NIF 10 chiffres + 2 lettres).

Recommandations prioritaires

- **[P0]** Étendre `FacturXExporter.writeTradeParty` pour sérialiser `nif` comme `SpecifiedTaxRegistration` avec `schemeID="NIF_HT"` quand `country='HT'`.
- **[P0]** Ajouter `openpdf` au classpath (`invoicing/build.gradle.kts`) et finaliser `embedFacturXInPdf` (le PDF/A-3 est requis pour toute facturation électronique conforme).
- **[P1]** Créer des templates facture/avoir distincts par `country` : template FR (CGI art. 289), template HT (Code Fiscal art. 196).
- **[P1]** Valider format NIF haïtien (10 chiffres + 2 lettres : `^[0-9]{10}[A-Z]{2}$`) dans `UpdateCompanyLegalFieldsRequest` si `country='HT'`.
- **[P2]** Évaluer la signature électronique exigée par la DGI Haïti (arrêté 2017) — actuellement aucune signature `XMLDSig/XAdES` n'est implémentée.

6.8 Déclarations fiscales et calendrier DGI

C'est probablement le point le plus critique de l'audit fiscal. Le service `TaxService.getDeclarationSchedule()` (lignes 426-506) génère un calendrier déclaratif **100% français** : TVA mensuelle au 19 du mois M+1 (12 échéances), TVA trimestrielle (4 échéances), IS acomptes aux 15 mars/juin/sept/déc (article 1668 CGI), IS solde au 15 mai N+1, DES mensuelle intra-UE au 10 du mois M+1 (article 289 B CGI).

La recherche de chaînes « DCR », « DCRf », « DCRG », « DCLS », « DGI » dans les modules `tax/`, `reporting/`, `financial-statements/` retourne **zéro résultat**. Aucun équivalent DGI haïtien n'est implémenté. Le calendrier DGI haïtien réel est pourtant très différent :

- **Mensuel (15 du mois M+1)** : TVA, TCA, RS, Acompte IS 1% sur encaissements (art. 5 Code Fiscal).
- **Annuel (31 mars N+1)** : IS (solde + DCR + DCRf + DCRG), DCLS (états financiers).

L'utilisateur qui suivrait le calendrier généré par le SaaS serait donc systématiquement en retard de déclaration (15 mai au lieu de 31 mars pour l'IS annuel ; absence totale des déclarations mensuelles TCA, RS et acompte IS). Sanctions Code Fiscal art. 240 : pénalités 10% à 50% du montant dû + intérêts de retard. Le service `TaxExportService` exporte uniquement le format CA3 français ; DES et EFI renvoient 501 Not Implemented.

Score déclarations DGI : 0 / 10 — Totalement absent

Aucun mécanisme déclaratif DGI n'est implémenté. Le calendrier est exclusivement français. Cette absence rend le SaaS inutilisable pour la conformité fiscale réelle d'une entreprise haïtienne, qui doit obligatoirement déclarer mensuellement la TVA, TCA, RS et l'acompte IS, et annuellement la DCR/DCRf au 31 mars. C'est le blockeur numéro un du déploiement haïtien.

Recommandations prioritaires

- [P0] Implémenter `HaitianTaxDeclarationSchedule` avec : TVA mensuelle (15 M+1), TCA mensuelle (15 M+1), RS mensuelle (15 M+1), Acompte IS mensuel 1% (15 M+1), DCR annuelle (31 mars N+1), DCRf annuelle (31 mars N+1).
- [P0] Implémenter `DgiHaitiExportService` pour les formats DCR (résumé), DCRf (feuille 1), DCRG (grand livre), DCLS (états financiers) au format Excel/CSV attendu par la DGI.
- [P1] Rendre `getDeclarationSchedule` contextuel au country (FR → CGI, HT → Code Fiscal, CA → CRA).
- [P2] Suivi des échéances avec notifications (module notifications) 7 jours avant chaque échéance DGI.

6.9 Synthèse fiscale Haïti — Tableau récapitulatif

Module	Maturité Haïti (0-10)	Écarts critiques
chart-of-accounts (PCN_HAITI)	4/10	PCN traité comme SYSCOHADA renommé ; comptes 442/446/447/4438 non seedés
tax (TVA, TCA)	3/10	Aucune TCA ; reverse-charge UE-only ; crédit TVA non persisté
tax (IS)	2/10	Fallback hardcodé France 25%/15%/42500€ ; calendrier 15 mai N+1
tax (RS + cotisations)	5/10	Architecture OK ; aucune règle HT seedée (CNSS/OFATMA/AST)
payroll	2/10	Constantes hardcodées FR (173.33h, HS 25/50%, 30j/mois) ; bulletin 100% C. trav. FR
financial-statements	3/10	Bilan/CR/CF universels ; pas de TAFIRE réel ; pas d'export DGI
invoicing (Factur-X)	3/10	XML Factur-X sans NIF HT ; mentions légales 100% CGI ; PDF/A-3 non implémenté
tax (déclarations)	0/10	Calendrier 100% français (CA3, DES, EFI) ; aucune DCR/DCRf/DCRG/DCLS
bank-reconciliation	6/10	CSV/OFX/MT940 OK ; pas de directory banques HT ; MT940 absent enum
fx-operations	8/10	HTG/USD complet ; manque réévaluation auto en clôture
funds-grants	6/10	Subventions/restrictions OK ; pas de formats bailleur (USAID/BM)
fixed-assets	3/10	Amortissement linéaire/dégressif ; pas de taux officiels HT/SYSCOHADA ; bug M11
employees	3/10	ContractType FR (CDI/CDD/Consultant) ; pas CNSS/OFATMA/matricule HT

Module	Maturité Haïti (0-10)	Écarts critiques
document-generation	2/10	Mentions légales 100% françaises (CGI/C. com./C. trav.)
reporting	4/10	15 exports CSV/PDF génériques ; aucun format DGI Haïti
accounting-engine	6/10	Moteur à double partie robuste ; clôture OK ; pas de rééval auto
company	5/10	NIF stocké ; LegalForm FR (SARL/SA/SAS) ; pays non utilisé pour spécialisation

6.10 Top 5 des risques fiscaux pour un déploiement Haïti

[P0] IS calculé au taux français par défaut

Si une entreprise haïtienne ne configure pas de CorporateTaxRule, son IS est calculé à 25% au lieu de 30%. Redressement DGI systématique + pénalités 50% + intérêts 1%/mois.

[P0] Calendrier déclaratif inadapté

Le calendrier propose 15 mai N+1 (IS solde FR) au lieu de 31 mars N+1 (DCR HT) et ignore les obligations mensuelles DGI (TVA, TCA, RS, acompte 1% IS). Pénalités Code Fiscal art. 240 (10% à 50% du montant dû).

[P0] Aucune TCA modélisée

La TCA haïtienne (Code Fiscal art. 196 — 2% banques, 5% télécoms, 10% services) est obligatoire pour les secteurs banque/télécom/services. Sans TCA, sous-déclaration systématique → redressement + pénalités 50% + intérêts.

[P1] Mentions légales factures non conformes Code Fiscal art. 196

Les factures portent « CGI art. 289 + C. com. L441-10 + 40 EUR » au lieu des mentions haïtiennes. Sanction : amende 5 000 à 50 000 HTG par facture non conforme.

[P1] Bulletin de paie non conforme Code du Travail haïtien

PayrollCalculator calcule sur 173.33h/mois (FR 35h) au lieu de 208h (HT 48h), HS 25%/50% au lieu de 50%/100%, bulletin mentionne URSSAF au lieu de CNSS/OFATMA. Sanction : amende 1 000 à 5 000 HTG par bulletin + dommages-intérêts salarié.

Analyse architecture & ingénierie

L'architecture de JOAccountant Backend est l'un des points forts indéniables du projet. Le découpage en 30 modules Gradle organisés selon les principes du Domain-Driven Design stratégique, l'isolation compile-time garantie par 42 règles ArchUnit, l'adoption du pattern hexagonal via 5 ports dans le socle core, et la défense multi-tenant à trois niveaux (filtre HTTP + listener JPA + RLS PostgreSQL) témoignent d'une réelle maîtrise de l'ingénierie logicielle. Le verdict global de 7.5/10 reflète cette solidité, tempérée par quelques régressions tactiques à corriger.

7.1 Organisation des 30 modules Gradle

Le fichier `settings.gradle.kts` déclare 30 modules (29 si l'on exclut test-support qui est un module utilitaire). Chaque module suit une structure interne régulière : `controller` / `service` / `entity` / `repository` / `dto` / `event`. Cette régularité est exceptionnelle pour un projet de cette envergure et facilite grandement la navigation dans le code. 25 modules suivent exactement ce pattern, ce qui constitue une discipline d'équipe remarquable.

La répartition par couche est claire : 6 modules d'infrastructure pure (core, audit-trail, document-generation, notifications, test-support, app), 26 modules business dont 6 always-on (accounting, third-parties, invoicing, etc.) et 8 sectoriels activés par BusinessType (inventory, time-billing, funds-grants, etc.). Le mapping BusinessType → Modules est piloté par données (table `business_type_module`, seedée en V3_003), ce qui permet d'ajouter de nouveaux types métier sans redéploiement — excellent design.

Quelques ajustements seraient néanmoins souhaitables. Le module `:purchasing` et `:purchase-orders` ont un couplage fort (3-way match PurchaseOrder ↔ PurchaseInvoice) et pourraient être fusionnés en un module `:procurement` avec deux aggregates distincts. Le module `:employees` est très mince (1 entité) et pourrait être rattaché à `:payroll` ou `:third-parties`. À l'inverse, `:accounting-engine` cumule 5 aggregates distincts (FiscalYear, FiscalPeriod, Journal, JournalEntry, JournalLine + clôture + grand livre + balance) dans un même module — candidat à un découpage en `:fiscal-calendar` + `:journal` pour réduire le blast radius des changements.

7.2 Architecture multi-tenant — forces et limites

Le modèle multi-tenant retenu est **shared DB + shared schema + tenant_id discriminator** (vérifié dans `TenantHibernatePropertiesCustomizer.java` lignes 18-19). La colonne `company_id` sert de discriminant sur toutes les tables tenant-aware. La défense en profondeur se déploie sur trois niveaux complémentaires.

Niveau 1 — Filtre HTTP : `TenantClaimFilter` extrait le `companyId` du path `/api/v1/companies/{companyId}/...`, valide l'appartenance via le claim JWT `companies` (liste de `{companyId, role}`). En cas d'accès non autorisé, l'API renvoie 404 (pas 403) pour empêcher l'énumération d'IDs. Le filtre `TenantContextFilter` gère la corrélation (X-Correlation-Id) et le MDC pour les logs.

Niveau 2 — Listeners JPA : `TenantAwareEntityListener` (@PrePersist/@PreUpdate) stamp `companyId` depuis `TenantContext` si null, et lève une `ValidationException` si les deux sont null. `TenantDefenseInDepthListener` valide que `entity.companyId == TenantContext.getCompanyId()` pour empêcher les INSERT/UPDATE cross-tenant. Ces listeners ne protègent **pas contre les SELECT non scopés** — c'est leur principale limite.

Niveau 3 — Row Level Security PostgreSQL : la migration V51 active RLS sur 6 tables critiques (journal_line, journal_entry, sales_invoice, purchase_invoice, third_party, expense_report) avec policy tenant_isolation basée sur current_setting('app.current_tenant', true)::uuid. **Mais c'est ici que se trouve le bug critique** : aucun code Java n'exécute jamais SET LOCAL app.current_tenant = ? au début des transactions. La migration elle-même l'admet (lignes 27-30) : « Tant que ce câblage n'est pas en place, RLS reste armed but inactive ». En l'état, soit RLS est activé et toutes les requêtes retournent 0 ligne (fail-closed → application cassée), soit RLS est ignoré et il n'y a aucune défense au niveau DB.

Action P0 — Câbler RLS PostgreSQL côté Java

Implémenter un ConnectionCustomizer Hibernate (ou un filtre qui exécute SET LOCAL app.current_tenant = ? au début de chaque transaction avec TenantContext.getCompanyId()). Donner BYPASSRLS au user Flyway pour que les migrations puissent s'exécuter. Sans cette action, la défense en profondeur multi-tenant au niveau DB reste inerte, et toute la sécurité repose sur la discipline manuelle des développeurs dans les 50+ services qui appellent findById() sans filtre tenant.

7.3 Domain-Driven Design — stratégique vs tactique

Le DDD stratégique est exemplaire : chaque module Gradle = un bounded context, avec un périmètre métier clairement délimité et documenté dans un README dédié. Le mapping BusinessType → Modules piloté par données permet l'extension sans redéploiement. Les domain events (28 fichiers d'événements recensés, records Java immuables) sont publiés via ApplicationEventPublisher et consommés par des listeners async (AuditEventListener, ForensicEventListener, DomainEventListener).

Le pattern hexagonal est également bien appliqué : 5 ports dans core/port/ (FileStoragePort, NotificationChannelPort, TaxRulePort, WithholdingRulePort, ApproverEmailResolverPort) avec implémentations swappables via @ConditionalOnProperty. Le pattern est notamment utilisé pour casser une dépendance circulaire entre :invoicing et :tax (TaxRulePort dans :core permet à :invoicing de consommer les règles de TVA sans dépendre compile-time de :tax).

Cependant, le **DDD tactique est largement absent**. Les entités JPA sont plates et anémiques : SalesInvoice (170 lignes) est composé à 100% de getters/setters, sans aucune méthode métier. Les méthodes issue(), applyPayment(), void() qui devraient encapsuler les transitions d'état sont absentes — toute la logique vit dans les services (transaction scripts). Les aggregates ne sont pas modélisés : JournalEntry et JournalLine sont deux entités liées sans invariant d'aggregate, l'équilibre débit=crédit étant vérifié par trigger DB (V36) + service + listener JPA — trois endroits différents, source potentielle de divergence.

7.4 Dépendances et règles ArchUnit

Le graphe de dépendances entre modules est propre : aucun cycle détecté, vérifié par lecture croisée des 29 build.gradle.kts. Le socle :core est pur (aucun project(":...") dans son build.gradle.kts, vérifié par ArchUnit Rule 2). :audit-trail ne dépend que de :core. :app est consommateur final (feuille).

Quelques god modules sont néanmoins à surveiller : :reporting dépend de 19 modules (justifié pour un agrégateur transverse mais indique une violation potentielle du principe of common closure), :payroll dépend de 11 modules (calcul brut→net nécessite tax + employees + accounting + document-generation), :invoicing dépend de 10 modules. Ces fan-in élevés sont acceptables mais justifient une vigilance accrue lors des refactors.

Les 42 règles ArchUnit (`ArchUnitTest.java`) couvrent : isolation du socle, dépendances amont/aval par module, entités `TenantAwareEntity`, contrats par phase. **Une incohérence critique a été détectée** : la Règle 28 interdit à `:notifications` de dépendre de `:invoicing`, `:accounting-engine`, `:bank-reconciliation`, `:fixed-assets`, `:chart-of-accounts` — mais `notifications/build.gradle.kts` déclare ces 5 dépendances et `DomainEventListener.java` importe explicitement 9 events de ces modules. Le test devrait échouer ; soit il y a un bug d'exécution, soit ArchUnit a un comportement inattendu. **À vérifier en priorité P0.**

7.5 Patterns architecturaux et API design

Le pattern Transaction Script domine (vs Domain Model) — choix assumé pour un MVP. CQRS et event sourcing sont absents, ce qui est raisonnable pour la phase actuelle du projet. Spring Batch est utilisé pour deux jobs : `payrollJob` (chunk-oriented taille 50, retry 3× sur `IOException`, listener de totaux) et `fiscalYearClosingJob` (tasklet idempotent, retry 1×). Le schéma `BATCH_*` est créé en V52. Le choix Spring Batch est justifié pour les workloads batch (paie de 500+ employés, clôture annuelle).

L'API design est solide : 190 endpoints REST sur 32 contrôleurs, préfixe `/api/v1` cohérent, `ProblemDetail` RFC 7807 implémenté dans `GlobalExceptionHandler` (404 pour `NotFoundException`, 409 pour `ConflictException`, 422 pour `ValidationException`, etc.) avec extensions `code`, `correlationId`, `companyId`, `timestamp`. OpenAPI est configuré avec 28 tags, 11 `GroupedOpenApi` par module, 3 serveurs (dev/staging/prod).

Le rate limiting est implémenté via `Bucket4j` 8.10.1 in-memory (10 req/min/IP sur `/login`, `/forgot-password`, etc. + `lockout` 5 échecs / 15 min sur couple IP+email). **Limitation majeure** : le store est in-memory (`ConcurrentHashMap<Bucket>`), donc incompatible multi-instances — 3 replicas K8s = 3× la limite effective. La migration vers `bucket4j-redis` est mentionnée en TODO v4.8 mais non implémentée.

7.6 Base de données et migrations Flyway

Les 63 migrations Flyway (V0_000 → V52) témoignent d'un travail d'ingénierie DB soigné. Aucun `DROP COLUMN/DROP TABLE/TRUNCATE` destructif n'a été trouvé (vérifié par `grep`) — politique additive stricte. Les triggers statement-level (V36 `trg_journal_entry_balance_*`) vérifient l'équilibre débit=crédit avec transition tables, complexité $O(N)$ au lieu de $O(N^2)$ — optimisation remarquable.

Les index sont bien pensés : 146 `CREATE INDEX` au total, avec index partiels pertinents (`WHERE status='POSTED'`, `WHERE matched=FALSE`, `WHERE third_party_id IS NOT NULL`) qui réduisent la taille des indexes. Tous les index composites commencent par `company_id` (pattern tenant discriminator first) — conforme aux bonnes pratiques multi-tenant.

Deux limites à signaler. Premièrement, la **numérotation des migrations est incohérente** : deux schémas coexistent (V<phase>_<seq> comme V7_001, et V<seq> simple comme V23). Flyway accepte car la comparaison est lexicale, mais c'est source d'erreur potentielle. Deuxièmement, `CREATE INDEX CONCURRENTLY` n'est jamais utilisé (justifié : Flyway s'exécute en transaction → `CONCURRENTLY` interdit), mais pour les migrations hot sur DB chargée, les index manquants ajoutés ultérieurement bloqueront la table — il faudra un script de pre-deploy hors Flyway pour les ajouts d'index en production.

7.7 Verdict architecture : 7.5 / 10

L'architecture de JOAccountant est solide, mature et bien documentée. Elle se distingue par un découpage DDD stratégique exemplaire, une défense multi-tenant à 3 niveaux, une stack moderne et

une observabilité complète. Les principaux freins pour atteindre 9/10 sont : RLS V51 non câblé côté Java, anemic domain model, Java 17 au lieu de 21, tests 100% intégration, et DevOps incomplet. Avec les 10 actions prioritaires (notamment les P0 : RLS câblage, Java 21, JaCoCo vérifié, TaskDecorator async), l'architecture passerait à 8.5/10 en moins de 2 semaines d'effort.

Analyse sécurité applicative

L'audit sécurité révèle un projet qui a investi dans les fondations cryptographiques (Argon2id conforme OWASP 2024, MFA TOTP RFC 6238, JWKS RFC 7517, RefreshToken rotation avec détection de réutilisation) mais qui présente **trois vulnérabilités critiques d'authentification et de gestion des secrets** qui doivent être corrigées avant tout go-live. Le score global sécurité de 5.9/10 reflète ce paradoxe : des fondations solides sapées par des bugs critiques sur des fonctionnalités mises en avant comme différenciateurs (MFA, RLS, audit forensique).

8.1 Authentification — 3 vulnérabilités critiques

L'authentification combine JWT (HS256 par défaut, RS256 supporté), MFA TOTP RFC 6238, Argon2id, RefreshToken rotation et PasswordResetToken single-use. L'implémentation Argon2id est conforme aux minimums OWASP 2024 : 19 MiB memory, 2 iterations, parallelism=1, hash length 32 bytes, salt 16 bytes SecureRandom, comparaison constante-time. Le password validator impose min 12 caractères (configurable), 1 majuscule, 1 minuscule, 1 chiffre, 1 spécial, avec blacklist de 50 mots de passe courants.

Vulnérabilité critique #1 — Bypass MFA : le endpoint `/auth/login/mfa` (`AuthController.loginMfa()` lignes 198-221) valide le `mfaChallengeToken` via `JwtService.parseClaims()` (lignes 236-248) qui appelle `SignedJWT.parse(jwt)` **sans vérifier la signature**. Un attaquant peut forger un JWT non signé avec `sub=<userId victime> + companies=[...]`, puis brute-forcer le code TOTP (1M combinaisons, fenêtre ± 1 période = 3 codes valides à un instant t). Le endpoint `/auth/login/mfa` n'est pas rate-limité par `RateLimitFilter.isAuthEndpoint()` — une omission critique. **Impact : prise de compte complète d'un utilisateur MFA-activé.**

Vulnérabilité P0 — Bypass authentification MFA

Fichiers : `auth/service/JwtService.java:236-248`,
`auth/controller/AuthController.java:198-221`.

Correction : valider la signature du `mfaChallengeToken` via `JwtDecoder Spring` (pas `parseClaims()`), passer le token en header `Authorization: Bearer` ou en body (pas en query param — fuite dans logs nginx/Tomcat), étendre `RateLimitFilter` à `/auth/login/mfa`.

Vulnérabilité critique #2 — Clé MFA sans fail-fast : la propriété `app.mfa.encryption-key` a une valeur par défaut `dev-only-mfa-key-please-override-32-chars` committée dans Git (`MfaService.java:71`). Contrairement à `JwtService.validateSecret()` qui refuse ce pattern en profil non-dev/test, **aucun contrôle de démarrage n'existe pour la clé MFA**. Si la variable d'environnement `APP_MFA_ENCRYPTION_KEY` n'est pas positionnée en production, l'application démarre silencieusement avec une clé publique — tous les secrets TOTP des utilisateurs sont alors chiffrés avec une clé connue de l'attaquant. Un attaquant obtenant un accès à la base de données (leak, backup compromis) peut déchiffrer tous les `mfa_secret.secret_encrypted` et forger des codes TOTP.

Vulnérabilité critique #3 — RLS inactif : traitée en détail dans la section 7.2. La migration V51 active Row Level Security sur 6 tables critiques mais aucun code Java n'exécute `SET LOCAL app.current_tenant = ?`. La défense en profondeur au niveau DB est inerte — toute la sécurité multi-tenant repose sur la discipline manuelle des développeurs dans les 50+ services qui appellent `findById()` sans filtre tenant.

8.2 Autorisation, multi-tenant et OWASP Top 10

Le RBAC est granulaire avec 6 rôles par société : OWNER > ADMIN > ACCOUNTANT > BOOKKEEPER > VIEWER > AUDITOR. L'entité `UserCompanyRole` (V2_002) associe un rôle par couple (`userId`, `companyId`) avec `acceptedAt=null` jusqu'à acceptation de l'invitation. Le `RoleChecker.ensureRole()` lit le claim JWT `companies` en stateless (pas de DB hit). L'auto-attribution du rôle OWNER est interdite (`OWNER_NOT_INVITABLE`). Le workflow d'approbation « quatre yeux » (`ApprovalWorkflowService.ensureFourEyes()`) empêche l'auto-approbation (403 `SELF_APPROVAL_FORBIDDEN` si `requestedBy == decidedBy`).

Cependant, **aucune annotation `@PreAuthorize` / `@Secured` n'est utilisée** — tout est `roleChecker.ensureRole()` explicite en début de chaque méthode de contrôleur (110+ occurrences). Risque : oubli sur un endpoint = IDOR silencieux. La couverture OWASP Top 10 est partielle :

Catégorie OWASP	Statut	Détails
A01 Broken Access Control	Partiel	RBAC programmatique ; RLS V51 inactif ; 50+ <code>findById</code> non scopés
A02 Cryptographic Failures	Partiel	Argon2id OK, AES-256-GCM OK, mais <code>APP_MFA_ENCRYPTION_KEY</code> sans fail-fast ; HS256 défaut ; pas de rotation JWKS
A03 Injection	Couvert	JPQL paramétré partout, pas de SQL natif concaténé, XXE protection OFX2
A04 Insecure Design	Partiel	Audit trail non immutable, MFA login 2-step avec <code>parseClaims()</code> non sécurisé
A05 Security Misconfiguration	Partiel	CORS * par défaut avec <code>allowCredentials=true</code> (combiné dangereux)
A06 Vulnerable Components	Manquant	Pas de Dependabot, Snyk, Trivy, OWASP Dependency-Check
A07 Identification & Auth Failures	Partiel	<code>parseClaims()</code> non signé pour MFA, <code>/auth/login/mfa</code> non rate-limité, lockout in-memory non distribué
A08 Software/Data Integrity	Manquant	Pas de Cosign (signature image), pas de SBOM (CycloneDX/Syft)
A09 Security Logging/Monitoring	Couvert	13 types d'événements sécurité tracés, <code>PiiMasker</code> , <code>AFTER_COMPLETION</code> , logs JSON
A10 SSRF	Faible risque	Pas d'appels HTTP sortants côté serveur

8.3 Audit trail — non immutabilité critique

L'audit trail est implémenté via `AuditLog` (11 colonnes : `id`, `company_id`, `actor_user_id`, `entity_type`, `entity_id`, `action`, `old_value_json`, `new_value_json`, `occurred_at`, `correlation_id`, `version`) avec 3 index pour la forensique. `AuditEventListener` est `@Async` + `@TransactionalEventListener(AFTER_COMMIT)` — bug corrigé v4.7 §5.1 Finding #2 (re-publication via `ApplicationEventPublisher` pour réactiver le proxy). `SecurityAuditEventListener` utilise `AFTER_COMPLETION` pour tracer les événements même sur transactions roll back (`LOGIN_FAILED`, `REFRESH_TOKEN_REUSED`) — excellent design.

Cependant, **la table `audit_log` n'est pas immutable**. Aucun trigger `BEFORE UPDATE/DELETE` n'empêche la modification ou la suppression des lignes. Aucun `REVOKE UPDATE, DELETE` sur le rôle applicatif. Un attaquant (ou un admin DB négligent) peut altérer ou supprimer l'audit forensique. Pour un SaaS comptaible soumis au Code Fiscal Haïtien (10 ans rétention obligatoire) et au Décret du 23 mai 2018 sur les données personnelles, c'est une non-conformité légale majeure.

Le `PiiMasker` masque les données personnelles (email, fullName, phone) dans les JSON avant persistance, mais il est regex-based (pas un vrai parseur JSON) — il peut rater des PII dans des structures imbriquées. La conservation n'a pas de politique de rétention explicite au niveau DB (mention 10 ans dans OPERATIONS.md mais pour les backups, pas pour la table elle-même). Pas de WORM (Write Once Read Many).

8.4 Gestion des secrets et cryptographie

La gestion des secrets présente des contrasts. Côté JWT, le pattern est exemplaire : `JwtService.validateSecret()` refuse le pattern dev en profil non-dev/test et exige ≥ 32 caractères (256 bits). JWKS endpoint conforme RFC 7517. Côté MFA, le pattern est défaillant : pas de fail-fast sur `APP_MFA_ENCRYPTION_KEY`. Côté DB, les credentials par défaut `joaccountant/joaccountant` sont acceptés silencieusement en prod.

Aucune intégration Vault / AWS Secrets Manager / Spring Cloud Config n'est présente dans le code. Le `helm/README.md` mentionne External Secrets Operator / Vault comme recommandation, mais le chart fournit seulement un Secret K8s chart-managed (base64, non chiffré au repos sauf si etcd encryption at rest activé). La rotation des secrets est documentée (OPERATIONS.md § 7) mais pas automatisée, pas de JWKS multi-keys (rotation impossible sans casser les JWT en circulation), pas de procédure documentée pour `APP_MFA_ENCRYPTION_KEY` (rotation impossible sans re-chiffrer tous les `mfa_secret.secret_encrypted` — script non fourni).

8.5 Verdict sécurité : 5.9 / 10 — Insuffisant

Le backend JOAccountant v5.2 est un projet mature sur le plan fonctionnel avec des fondations sécurité solides (Argon2id, MFA TOTP, refresh rotation, audit forensique, PII masking, observabilité 3 piliers, backup PITR testé mensuellement). Cependant, il n'est PAS prêt pour la production haïtienne en l'état : 3 vulnérabilités critiques d'authentification/crypto (bypass MFA, clé MFA sans fail-fast, RLS inactif), audit trail non immutable (non-conformité Code Fiscal Haïtien), pipeline CI/CD sans scans sécurité, et conformité au Décret haïtien du 23 mai 2018 absente. Les 5 actions P0 listées dans la section 14 doivent être exécutées avant tout go-live.

Analyse DevOps, observabilité & ops

L'analyse DevOps révèle un projet avec une observabilité production-grade (Prometheus + OpenTelemetry + logs JSON structurés + MDC correlationId/userId/companyId) et un backup PITR testé mensuellement — rare et excellent pour un projet de cette taille. Mais le pipeline CI/CD est minimaliste (2 jobs sans scans sécurité) et le Helm chart K8s manque de NetworkPolicy, PodDisruptionBudget et ServiceMonitor. Le score global DevOps de 7/10 reflète ce contraste : excellent sur l'observabilité et le backup, insuffisant sur la CI/CD.

9.1 CI/CD — pipeline minimaliste sans scans sécurité

Le pipeline GitHub Actions (.github/workflows/ci.yml) comporte 2 jobs seulement : build-and-test (JDK 17, cache Gradle, ./gradlew check --continue, ./gradlew test, upload rapports, build JAR) et build-docker (build Docker image sans push sur PR). Aucun CD (continuous deployment) — pas de push vers registry, pas de deploy Helm.

Manquants critiques : pas de scan SAST (CodeQL, SonarQube, Semgrep), pas de scan SCA (Dependabot, Snyk, Trivy, OWASP Dependency-Check), pas de scan d'image Docker (Trivy, Gype), pas de signature d'image (Cosign, Notation), pas de SBOM (CycloneDX, Syft), pas de push d'image vers registry. SpotBugs et PMD sont configurés en local mais en mode warning-only (ignoreFailures = true) — l'analyse statique ne fail jamais le build. JaCoCo est configuré mais failOnViolation = false — le seuil 70% lignes / 60% branches n'est pas enforced.

Le risque est réel : une dépendance vulnérable (type Log4Shell) ou une image compromise peut atteindre la production sans détection. La dépendance Bouncy Castle 1.78.1 actuellement dans le projet présente 2 CVEs publics (CVE-2024-29857, CVE-2024-30171) qui ne seraient pas détectées sans scan SCA. Dependabot est activé (Gradle + GitHub Actions + Docker, hebdo) mais seul le catalog de versions est tracké — pas les CVEs transitives.

9.2 Observabilité — production-grade

L'observabilité est l'un des points forts du projet, déployée sur les 3 piliers classiques. **Logs** : Logback-spring.xml avec 2 profils (console lisible en dev, JSON via LogstashEncoder en prod) avec MDC [correlationId, userId, companyId]. Niveaux appropriés : jo.accountant=INFO, org.hibernate.SQL=WARN, org.springframework=WARN en prod.

Métriques : Micrometer + Prometheus exposé sur port 8081 séparé (excellent — sépare le trafic API du trafic ops). Percentiles HTTP P50/P90/P95/P99 activés avec histogram. Bindings auto : JVM (heap, threads, GC, classloader), HikariCP (connexions actives/idles, wait, leaks), HTTP server (requêtes, latence), Process (CPU, mémoire). BusinessMetricsConfig custom pour les métriques métier. ShedLock pour empêcher la double-exécution des crons multi-instances via table DB partagée.

Tracing : OpenTelemetry (micrometer-tracing-bridge-otel + opentelemetry-exporter-otlp) avec export OTLP gRPC vers OTEL_EXPORTER_OTLP_ENDPOINT. Sampling ParentBased(TraceIdRatioBased) 10% par défaut en prod. deploy/otel-collector-config.yaml fourni pour test local. **Limitation** : sans OTEL_EXPORTER_OTLP_ENDPOINT configuré, le tracing est inactif en prod par défaut (traces dropped silencieusement, pas d'erreur).

Health checks : Actuator sur port 8081 séparé, management.endpoint.health.probes.enabled=true (liveness + readiness). Helm probes.liveness.path=/actuator/health/liveness, probes.readiness.path=/actuator/health/readiness,

`probes.startup.path=/actuator/health` — conforme K8s.

Manquants : aucun dashboard Grafana fourni (juste annotation `prometheus.io/scrape: "true"`), aucune règle Alertmanager formelle (`OPERATIONS.md §8` décrit des runbooks par symptôme mais sans règles Prometheus), aucune SLO formelle (Service Level Objective). Pas de Sentry ou équivalent pour error tracking (mention TODO §11 `OPERATIONS.md`).

9.3 Backup & DR — rare et excellent

Le dispositif de backup et disaster recovery est exceptionnel pour un projet de cette taille. Le script `scripts/backup-postgres.sh` utilise `pg_basebackup full + WAL archiving continu (PITR — Point-in-Time Recovery)` avec compression `gzip -Z 6 + checksum SHA-256`. Sync S3 cross-region avec SSE-KMS si `S3_BACKUP_BUCKET` configuré. Rotation : 7 daily + 4 weekly + 12 monthly + 10 yearly (conformité fiscale 10 ans).

Les RPO/RTO sont documentés (`OPERATIONS.md §4.2`) : **RPO < 5 min** (WAL streaming), **RTO < 1h** (`pgBackRest restore`). Le test de restauration est automatisé mensuellement via `scripts/test-restore.sh + scripts/cron-backup-test.sh` avec smoke test complet : comptage users, équilibre débit/crédit des `journal_entry POSTED`, FK violations, présence du trigger `trg_journal_entry_balance`. Cron 1er samedi du mois + notification Slack (succès/échec). **C'est rare** — la plupart des projets ne testent jamais leurs backups.

Runbook DR (`OPERATIONS.md §3`) : `rollback application via kubectl rollout undo`, `rollback DB via pgbackrest restore --type=time + flyway repair`, politique de migrations cassantes additive seulement (expand/contract). Manquants : pas de chaos engineering (chaos-mesh, litmus) documenté, pas de test de restauration cross-region, pas de topologie PostgreSQL multi-AZ documentée au niveau applicatif.

9.4 Dockerfile et Helm chart

Le Dockerfile est multi-stage (Stage 1 : `eclipse-temurin:21-jdk` pour build, Stage 2 : `eclipse-temurin:21-jre` pour runtime), avec user non-root (`joaccountant`), JVM args optimisés containers (`MaxRAMPercentage=75, G1GC, HeapDumpOnOutOfMemoryError`) et volume persistant `/var/lib/joaccountant/storage`.

Incohérence critique : le Dockerfile utilise JDK/JRE 21 mais `build.gradle.kts:48` force `JavaVersion.VERSION_17`. Le bytecode produit est Java 17 mais tourne sur JRE 21 → OK mais sous-utilise Java 21. L'option `spring.threads.virtual.enabled=true` dans `application.yml` est no-op en Java 17 (virtual threads stables en Java 21). Pas d'image distroless (utilise `eclipse-temurin:21-jre` basé sur Ubuntu ~250 MB — une distroless ferait ~80 MB).

Le Helm chart (`deploy/helm/joaccountant/`) est complet : HPA (2→10 replicas, CPU 70%, `scaleUp 60s / scaleDown 300s`), `probes liveness/readiness/startup`, PVC 20Gi, `PodSecurityContext runAsNonRoot: true, allowPrivilegeEscalation: false, capabilities.drop: [ALL]`, `RollingUpdate zero-downtime`, annotation `checksum/config` pour forcer le restart quand la `ConfigMap` change. **Manquants** : `NetworkPolicy` (isolation pods), `PodDisruptionBudget`, `ServiceMonitor` pour Prometheus Operator (annotation `prometheus.io/scrape` utilisée à la place — moins puissant), pas de Helm tests.

9.5 Verdict DevOps : 7 / 10 — Correct

Le dispositif DevOps est correct mais incomplet. L'observabilité et le backup PITR testé mensuellement sont exemplaires et constituent un avantage compétitif majeur. Mais le pipeline CI/CD minimaliste (2

jobs, pas de scans sécurité) et le Helm chart K8s à compléter (NetworkPolicy, PDB, ServiceMonitor) exposent le projet à des risques opérationnels en production. Les 5 actions P1 listées dans la section 14 (Trivy, Dependabot, CodeQL, Cosign, CD pipeline) sont nécessaires pour atteindre un niveau production-ready.

Analyse dette technique & qualité code

L'analyse qualité révèle un projet paradoxal : **une documentation exceptionnelle** (720 lignes README, 28 README par module, 197 annotations @Operation, OPERATIONS.md 317 lignes) et des conventions de code modernes (146 records Java 17, 0 Lombok, constructeur injection systématique, RFC 7807) qui témoignent d'une réelle rigueur d'équipe. Mais cette qualité documentaire contraste avec **une dette de test majeure** (27 modules métier à 0% de tests unitaires, 100% tests d'intégration lents, JaCoCo non bloquant) qui empêche tout refactoring serein. Le score global de 6.5/10 reflète ce ratio documentation/tests inversé, atypique et signe d'un projet en fin de build sans phase de durcissement.

10.1 Volumétrie et god classes

Le projet comporte **464 fichiers Java principaux** pour ~46 000 lignes de code, **31 fichiers de test** pour ~10 000 lignes, et **63 migrations Flyway** pour ~3 500 lignes SQL. Le ratio test/main de 22% est faible. Les 5 plus gros modules par taille sont accounting-engine (3 839 LOC), auth (2 839), fixed-assets (2 856), company (2 482) et tax (2 383).

18 fichiers dépassent 500 lignes (code smell potentiel), dont 8 particulièrement problématiques : FixedAssetsService (1 366 LOC, 11 dépendances injectées, méthodes de 165 lignes), InvoicingService (1 082 LOC), ReportingService (1 016 LOC, 14 modules, 37 champs injectés), AccountingEngineService (935 LOC, 20 champs injectés), PayrollService (822 LOC). Ces god classes constituent la principale dette technique du projet.

12 méthodes dépassent 80 lignes. Les 5 plus longues : FiscalYearClosingService.java:141-312 (171 lignes, clôture exercice fiscal), FixedAssetsService.java:765-930 (165 lignes, méthode dispose()), InvoicingService.java:333-496 (163 lignes, émission facture + écritures), PayrollService.java:165-301 (136 lignes), FixedAssetsService.java:626-744 (118 lignes). Ces méthodes sont des candidats évidents à l'extraction en méthodes privées plus courtes.

10.2 Couverture de tests — la plus grande faiblesse

La couverture de tests est le point le plus critique du projet. **282 @Test au total** (vs 310 annoncé — l'écart provient du comptage des @ParameterizedTest multiples) sur 31 fichiers, **tous dans app/src/test/java/jo/accountant/app/**. **0 fichier de test dans 27 modules sur 28** — seul le module app est testé. **0 mock Mockito** (grep @Mock/@MockBean = 0) → 100% des tests sont des tests d'intégration @SpringBootTest.

Dette critique — 27 modules à 0% de tests unitaires

Modules sans AUCUN test (test directory absente) : accounting-engine, analytics, approval-workflow, audit-trail, auth, bank-reconciliation, chart-of-accounts, company, core, document-generation, document-numbering, employees, expenses, financial-statements, fixed-assets, funds-grants, fx-operations, inventory, invoicing, notifications, payroll, purchase-orders, purchasing, reporting, tax, test-support, third-parties, time-billing.

Conséquence : tout refactoring est risqué car seul le scénario d'intégration couvre les règles métier. Un bug dans PayrollCalculator (calcul brut→net) ou TaxService (calcul IS) peut passer inaperçu si le scénario d'intégration ne le couvre pas. La boucle de feedback développeur est lente (10-30 min par exécution CI).

L'ArchUnit Rule 28 est probablement cassée : elle interdit à :notifications de dépendre de :invoicing, :accounting-engine, :bank-reconciliation, :fixed-assets, :chart-of-accounts

— mais `notifications/build.gradle.kts` déclare ces 5 dépendances et `DomainEventListener.java` importe explicitement 9 events de ces modules. Le test devrait échouer ; soit il y a un bug d'exécution, soit ArchUnit a un comportement inattendu. **À vérifier en priorité P0** — si le test ne fail pas, c'est potentiellement toute la grille ArchUnit qui est silencieusement inopérante.

Le point positif est l'utilisation de Zonky embedded-postgres dans test-support : tous les tests `@SpringBootTest` démarrent un vrai PostgreSQL in-process (pgcrypto, uuidv7(), planner réels — pas H2). Cela valide les migrations Flyway, les types PostgreSQL, la RLS V51. **C'est la seule vraie force du dispositif de test.** JaCoCo est configuré avec cible 70% lignes / 60% branches mais `failOnViolation = false` (commenté ligne 118) — le build ne casse jamais. Le rapport XML est généré pour SonarQube mais SonarQube n'est PAS intégré.

10.3 Conventions de code — modernes et rigoureuses

Les conventions de code sont exemplaires. **146 records Java 17** (tous les DTOs Request/Response sont des records) — réduit le boilerplate et garantit l'immuabilité. **0 Lombok** (grep lombok = 0) — choix assumé en faveur des records Java 17 et de l'injection par constructeur, aucune dépendance runtime. **391 champs private final** (aucun `@Autowired` sur champ) — constructor injection systématique, pattern Spring recommandé.

Le nommage est cohérent (camelCase méthodes/variables, PascalCase classes, UPPER_SNAKE_CASE constantes). La structure est hybride : module par feature + package par layer à l'intérieur (controller, service, repository, entity, dto, event). 25 modules suivent exactement ce pattern. JavaDoc sur 83% des fichiers (386/464). Optional et Streams utilisés de façon idiomatique (312 usages Optional, 124 usages Streams). 9 `@SuppressWarnings` total, dont 2 `@SuppressWarnings("unused")` dans `S3FileStorageAdapter` (code mort potentiel).

10.4 Code smells et TODOs critiques

38 commentaires TODO/FIXME/HACK au total (36 TODO, 0 FIXME, 0 HACK), dont 32 étiquetés "TODO v4.8" / "TODO v4.9" (backlog explicite — bonne pratique). Top modules : tax (17), auth (6), invoicing (5), app (4), notifications (2). Trois TODOs sont particulièrement critiques :

- `tax/TaxService.java:247` — `BigDecimal taxCreditCarriedForward = BigDecimal.ZERO;`
// TODO : récupérer depuis table dédiée. **Calcul d'IS potentiellement erroné** — crédit de TVA non reporté d'une période à l'autre.
- `tax/TaxService.java:315` — `BigDecimal charasseAddition = BigDecimal.ZERO;` // TODO v4.8 : calculer depuis les dividendes reçus. **IS probablement sous-évalué** pour les sociétés percevant des dividendes (régime Charasse français).
- `auth/MfaController.java:95` — `String email = "user-" + userId;` // TODO v4.8 : récupérer le vrai email depuis `UserRepository`. **MFA non fonctionnelle** — l'URL otpauth envoyée à l'authenticator contient un email fictif, et l'email de notification MFA n'arrive jamais au bon destinataire.

Magic numbers hardcodés problématiques : `PayrollCalculator.MONTHLY_LEGAL_HOURS = 173.33` (France 35h, devrait être configurable), `PayrollCalculator.WORKING_DAYS_PER_MONTH = 30` (devrait être configurable), `TaxService.java:393-395` — `setStandardRate(new BigDecimal("25"))`, `setReducedRate(new BigDecimal("15"))`, `setReducedRateThreshold(new BigDecimal("42500"))` — taux d'IS France hardcodé dans le code Java au lieu d'être en DB.

10.5 Dépendances externes — CVEs connus

L'audit des dépendances (via `gradle/libs.versions.toml`) révèle plusieurs versions à mettre à jour. **Bouncy Castle 1.78.1** présente 2 CVEs publics : CVE-2024-29857 (DoS via EACertificate) et CVE-2024-30171 (timing attack sur RSA decryption TLS). Doit être bumpé à 1.79+. **Caffeine 3.1.8** est ancien (dernière stable 3.2.x). **ShedLock 5.10.0** a une version courante 6.x. **Java 17** est dépassé par Java 21 LTS.

Incohérence Dockerfile/build : Dockerfile utilise JDK/JRE 21 mais `build.gradle.kts` force Java 17 — le bytecode Java 17 tourne sur JRE 21, ce qui fonctionne mais sous-utilise Java 21 (virtual threads configurés via `spring.threads.virtual.enabled=true` mais no-op en Java 17). La migration à Java 21 activerait les virtual threads nativement, découplant les threads Tomcat des connexions DB — gain de scalabilité majeur.

Spring Boot 3.5.0 est aligné sur la dernière version (mai 2024), Spring Framework 6.2, Jakarta EE 10, Hibernate 6.5. La dépendance Spring Boot BOM est bien utilisée (`implementation(platform(SpringBootPlugin.BOM_COORDINATES))`). JUnit Platform est forcée en 1.11.0 via `resolutionStrategy.force(...)` — bonne prévention des conflits. Nombre total de dépendances directes : ~17 (modéré, bien géré).

10.6 Documentation — exemplaire (9/10)

La documentation est l'un des points forts du projet. **README principal de 720 lignes** détaillant architecture, modules, référentiels, types métier, scripts de seed, build, configuration. **28 README par module** présents (un par module). Top : `accounting-engine/README.md` (361 lignes), `company/README.md` (329), `reporting/README.md` (305). **OPERATIONS.md 317 lignes** — runbook d'exploitation (architecture, déploiement, monitoring, backup, recovery).

Documentation API : **197 annotations @Operation** (pour 196 endpoints → couverture quasi totale), **341 @Schema** sur DTOs records, **482 @ApiResponse** (annotations multiples par endpoint). Swagger UI exposé sur `/swagger-ui.html` en dev, désactivé en prod. JavaDoc sur 83% des fichiers (386/464). Manquants : pas de diagrammes PlantUML/Mermaid dans le code source (quelques diagrammes ASCII dans OPERATIONS.md).

10.7 Build, lint et qualité statique

Le build Gradle est bien configuré : Gradle 8.10.2, parallel + cache + daemon activés, JVM args `-Xmx4096m`. Spotless (google-java-format 1.22.0) activé sur tous les subprojects. SpotBugs (effort MAX, MEDIUM, `ignoreFailures = true`) et PMD (bestpractices + errorprone, `ignoreFailures = true`) configurés mais en mode warning-only — ne fail jamais le build. **Checkstyle non configuré**, **ErrorProne non configuré**, **NullAway non configuré**. SonarQube / SonarCloud non intégré malgré le rapport XML JaCoCo préparé pour.

Tests de charge Gatling présents (`app/src/test/gatling/JoAccountantSimulation.scala`, 176 lignes, 3 scénarios : login→dashboard, list→create invoice, trial-balance→balance-sheet) mais **plugin Gatling Gradle non activé** — la simulation n'est probablement pas exécutable via Gradle (livrée en documentation seule). Tests de mutation PIT : aucun. Tests de sécurité : aucun test dédié (pas de test OWASP, pas de fuzzing, pas de test sur RateLimitFilter, pas de test SecurityConfig).

Résidus à nettoyer : `gradle.properties` contient des références Android (`android.nonTransitiveRClass=true`, `android.useAndroidX=true`) — résidus d'un template Gradle Android, sans effet mais doivent être nettoyés.

10.8 Verdict qualité : 6.5 / 10 — Moyen

Le projet présente des fondations excellentes (architecture, documentation, patterns Java 17, observabilité, RFC 7807, multi-tenant RLS) qui le placent au-dessus de la moyenne des SaaS B2B. Mais la dette de test majeure (27 modules à 0%, 0 mock, 0 test unitaire) empêche tout refactoring serein et fait chuter le score global de 2 points. La dette fonctionnelle ciblée mais critique (MFA cassée, calcul d'IS partiel, Factur-X sans PDF/A-3) sur des fonctionnalités mises en avant dans le README est préoccupante. La stack qualité est installée mais non bloquante (SpotBugs/PMD warnings only, JaCoCo non enforced, SonarCloud absent).

Analyse scalabilité & performance

L'analyse scalabilité/performance révèle un projet avec **une indexation PostgreSQL remarquable** (146 index, index partiels bien pensés, trigger statement-level performant) mais **une couche applicative en retrait** (N+1 applicatifs sur TaxService et loadJournalEntryResponse, agrégations Java au lieu de SQL GROUP BY, persistance 1-par-1 sans batching JDBC). Le score global de 6.5/10 reflète ce contraste. Sans les correctifs P0 ci-dessous, l'application tiendra 50 utilisateurs concurrents (sizing actuel du pool HikariCP) mais OOM/timeout garantis au-delà, principalement sur /audit-trail et les endpoints de reporting financier.

11.1 Indexation PostgreSQL — 8/10, la grande force

L'indexation PostgreSQL est l'un des points forts du projet : **146 CREATE INDEX au total** dans les migrations, concentrés dans 3 migrations d'audit perf (V7_002, V36, V38). Les **index partiels WHERE** sont particulièrement bien pensés : `idx_journal_entry_company_posted ... WHERE status='POSTED'`, `idx_sales_invoice_company_due_open ... WHERE status IN ('ISSUED', 'PARTIALLY_PAID')`, `idx_bank_statement_line_unmatched ... WHERE matched = FALSE`, `idx_audit_log_security_events ... WHERE entity_type = 'SecurityEvent'`. Ces index partiels réduisent la taille des indexes et accélèrent les requêtes réelles.

Tous les index composites commencent par `company_id` (pattern tenant discriminator first) — conforme aux bonnes pratiques multi-tenant. 38 UNIQUE constraints garantissent l'intégrité (idempotency keys, codes uniques par tenant, etc.). Le trigger statement-level `trg_journal_entry_balance_*` (V36) vérifie l'équilibre débit=crédit avec transition tables en complexité $O(N)$ au lieu de $O(N^2)$ — optimisation remarquable pour les écritures de paie de 500+ lignes.

Index manquants mineurs : `journal_entry.journal_id` (FK sans index), `journal_entry.reversal_of_entry_id` (lookup des renversements = seq scan), `bank_statement_line (company_id, line_date)`, `tb_timesheet_entry (company_id, entry_date)`. **Pas de partitionnement PostgreSQL** (grep `PARTITION BY = 0`) — V38 L15-20 mentionne le partitionnement mensuel d'`audit_log` comme roadmap, non implémenté. `CREATE INDEX CONCURRENTLY` jamais utilisé (justifié : Flyway s'exécute en transaction → `CONCURRENTLY` interdit) — pour les migrations hot sur DB chargée, il faudra un script de pre-deploy hors Flyway.

11.2 Pagination — risque OOM critique sur /audit-trail

La pagination est inégalement appliquée. Endpoints paginés (Spring Data Page<>) : `GET /journal-entries/paged`, `GET /journal-entries/search`, `GET /invoicing/invoices`, `GET /purchasing/invoices`, `GET /expenses`, `GET /notifications`, `GET /third-parties`. Hard cap 200 sur la size côté controller.

Risque P0 — OOM certain sur GET /audit-trail

Le endpoint `GET /audit-trail` (`AuditTrailController.list` L40-49) appelle `findByCompanyIdOrderByOccurredAtDesc(companyId)` qui charge **TOUT l'historique d'audit** d'un tenant. La migration V36 L182 mentionne explicitement « Volume attendu : 100M+ lignes sur 5 ans ». Sur un tenant actif, le premier appel à ce endpoint déclenchera un `OutOfMemoryError` certain. **Action P0 : paginer avec filtres**
`?page=&size=&entityType=&actorUserId=&from=&to=&`.

Autres endpoints non paginés qui chargent List<> complet : GET /journal-entries legacy, GET /employees, GET /approval-requests, GET /financial-statements/snapshots, GET /fx-operations. La pagination OFFSET classique (Spring Data PageRequest.of(page, size)) est utilisée — pas de keyset/seek pagination. Sur audit_log avec 10M lignes, OFFSET 900000 LIMIT 50 = seq scan jusqu'à la 900 000e ligne. La keyset pagination (WHERE (entry_date, id) < (:from_date, :from_id)) serait 10× plus rapide sur les pages profondes.

11.3 Requêtes JPA et N+1 applicatifs

L'architecture JPA est plate (aucune relation @ManyToOne/@OneToMany, modèles UUID en colonnes simples) — ce qui élimine les N+1 classiques Hibernate liés au lazy loading. Cependant, 5 N+1 applicatifs majeurs ont été identifiés dans les services :

Service	Problème	Impact
AccountingEngineService.listJournalEntries (L733-737)	.map(e -> loadJournalEntryResponse(companyId, e.getId())) — chaque entrée re-fait 1+N requêtes	Page de 50 = ~350 requêtes
TaxService.getDeclaration (L167-264)	Boucle sur chaque facture : invoiceLineRepository.findInvoiceLineRepository.findById(inv.getId())	1000 factures = 1003 requêtes
BankReconciliationService.autoMatch (L371-424)	Pour chaque BSL unmatched, appelle journalLineRepository.findLedger(...)	5000 BSL unmatched = 5000 requêtes
FinancialStatementsService.getCashFlowStatement (L466-499)	findAllPostedBetweenDates + 2× findAllPostedUpToDate (opening + closing) → 3× toutes les lignes en mémoire	100K+ lignes → latence P99 > 10s, heap > 100MB
AccountingEngineService.getTrialBalance (L858-885)	Charge toutes les lignes puis accountRepository.findById() par compte final	N+1 sur le résultat

Ces N+1 sont des candidats évidents à la réécriture en SQL agrégé. Par exemple, TaxService.getDeclaration devrait être réécrit en une seule requête SELECT l.tax_rate, SUM(l.line_total_ht), SUM(l.line_total_tax) FROM invoice_line l JOIN sales_invoice s ON s.id = l.invoice_id WHERE s.company_id = :companyId AND s.status IN (...) AND s.issue_date BETWEEN :from AND :to GROUP BY l.tax_rate. Gain estimé : 100× plus rapide sur 1000 factures. De même, getBalanceSheet devrait utiliser SELECT account_id, SUM(debit), SUM(credit) ... GROUP BY au lieu de charger 100K+ entités en mémoire et agréger en Java.

11.4 Multi-tenant à grande échelle

La stratégie discriminator (shared DB + shared schema + tenant_id) est adaptée à un SaaS comptable avec ~1000-5000 tenants. Au-delà, le partitionnement par tenant ou par période devient nécessaire. @TenantId Hibernate 6 non activé — le listener TenantDefenseInDepthListener ne valide que INSERT/UPDATE, pas SELECT (IDOR reposant sur la discipline manuelle). RLS PostgreSQL activé sur 6 tables critiques mais câblage Java incomplet (cf section 7.2).

Pool HikariCP = 30 global (application.yml L11), sizing documenté pour 50 utilisateurs concurrents. Pour 1000+ users × 3 replicas : 30 × 3 = 90 connexions → en dessous du besoin réel. Commentaire L9-10 : « Pool Hikari=10 incompatible avec Virtual Threads (starvation systémique au-delà de 50 users). Passé à 30 ». HPA configuré 2→10 replicas, soit 10 × 30 = 300 connexions max vers PostgreSQL. Sur une DB PG avec max_connections=100 par défaut → **DB saturée avant l'app**. Solution : PgBouncer en mode transaction pooling devant PostgreSQL, ou montée à max_connections PG ≥ 500.

1000+ sociétés : tables partagées (audit_log, users, user_company_role, refresh_token, password_reset_token) croissent avec tous les tenants. Aucune segmentation. Pas de partitionnement

par tenant ni par période. `TenantContext ThreadLocal` simple (pas `InheritableThreadLocal`) — incompatible avec `@Async` (risque de perte de contexte tenant) et avec `virtual threads Java 21` (`ScopedValue` serait préférable).

11.5 Cache, batch et connection pooling

Cache Caffeine local configuré avec 8 caches nommés : `accounts` (TTL 30min, 5000 entrées), `journals` (30min, 1000), `taxRules` (10min, 1000), `withholdingRules` (10min, 1000), `corporateTaxRules` (10min, 1000), `accountingFrameworks` (1h, 100), `businessTypes` (1h, 100), `currencies` (1h, 200). `recordStats()` activé sur tous. Cache key namespacing par tenant (`companyId.toString() + ':' + ...`) — bonne pratique pour éviter fuite cross-tenant.

Limitation majeure : pas de cache distribué Redis (mention TODO v4.8 dans `CacheConfig L23-24`). En multi-instances (HPA 2→10 replicas), chaque instance a son cache → staleness jusqu'au TTL (30min pour `accounts`). Invalidation via `@CacheEvict(value = "accounts", allEntries = true)` — éviction totale du cache à chaque mutation d'un compte (réchauffe froide).

Spring Batch bien configuré : `payrollJob` chunk-oriented taille 50, `JpaPagingItemReader`, retry 3× sur `IOException`, `@StepScope`, listener de totaux. `fiscalYearClosingJob` tasklet idempotent avec retry 1×. Schéma `BATCH_*` créé en V52. `ShedLock` activé pour empêcher la double-exécution en multi-instances.

Imports bancaires : `BankReconciliationService.importStatement (L156-163)` persiste 1-par-1 `lineRepository.save(line)` dans une boucle for — pour 5000 lignes, 5000 INSERTs sans batching (pas de `hibernate.jdbc.batch_size` configuré). Doit être remplacé par `saveAll()` en batches de 100 + activation `hibernate.jdbc.batch_size=50`.

Génération PDF synchrone : `DocumentGenerationService.generateDocument (L117, openhtmltopdf)` n'est pas `@Async`. Appelé par `InvoicingService`, `PayrollService`, `ReportingService`. Sur génération de 1000 factures PDF batch → blocage thread Tomcat. **Calcul bilans non incrémental** : `FinancialStatementsService.getBalanceSheet` recharge toutes les lignes POSTED ≤ asOf à chaque appel. Les snapshots figés existent (`createSnapshot`) mais ne sont pas utilisés comme cache.

11.6 Tests de charge Gatling et SLO

Tests de charge Gatling présents : `app/src/test/gatling/jo/accountant/JoAccountantSimulation.scala` (176 lignes, 3 scénarios) avec SLO intégrés au build CI : P95 < 1s sur tous les endpoints lecture, P99 < 2s sur les agrégations reporting, erreur < 0.1%. Charge cible affichée : 10+2+5 = 17 users/sec soutenus, soit ~1000 users/sec pour la prod (avec HPA 10 replicas) — cohérent avec la cible 1000+ users si HPA actif. **Limitation** : plugin Gatling Gradle non activé (commentaire L37-38 du fichier `.scala`) — la simulation n'est probablement pas exécutable via Gradle.

Métriques Prometheus (`application.yml L80-89`) : percentiles HTTP P50/P90/P95/P99 activés avec histogram, HikariCP metrics, JVM metrics. Tag `application` sur toutes les métriques. **Manquants** : pas de scénario de charge sur `/audit-trail` (le pire cas OOM), pas de scénario sur `/financial-statements/cash-flow` (3× `findAllPosted`), pas de test de soak 24h pour détecter les fuites mémoire, pas de profiling JFR/async-profiler intégré au CI.

11.7 Optimisations Spring Boot

Plusieurs optimisations Spring Boot manquantes auraient un impact significatif. **HTTP/2 non activé** (pas de `server.http2.enabled: true`) — gain potentiel 30% sur latence multi-requêtes parallèles du mobile. **Compression gzip/brotli non configurée** (`server.compression.enabled` absent) — pour des

payloads JSON de 50K+ (audit-trail, listes), c'est dommage. Bande passante mobile réduite de 5-10× avec gzip.

ThreadPoolTaskExecutor @Async non configuré — @EnableAsync activé mais pas de TaskExecutor explicite → SimpleAsyncTaskExecutor par défaut (1 thread par appel @Async). Sur forte charge, explosion de threads. @Async utilisé sur 11 méthodes (AuditEventListener, SecurityAuditEventListener, DomainEventListener ×8, ForensicEventListener ×4). Doit être remplacé par un ThreadPoolTaskExecutor borné (corePoolSize=10, maxPoolSize=50, queueCapacity=100).

Bonnes pratiques déjà en place : `spring.jpa.open-in-view: false` (évite le lazy loading hors transactions), `spring.threads.virtual.enabled: true` (no-op en Java 17 mais prêt pour Java 21), `server.shutdown: graceful + lifecycle.timeout-per-shutdown-phase: 30s` (déploiements zero-downtime), `management.server.port 8081` séparé du port 8080 API.

11.8 Verdict scalabilité : 6.5 / 10 — Moyen

Les fondations DB sont solides (indexation 8/10, trigger statement-level, RLS PostgreSQL, cache Caffeine avec stats). Mais la couche applicative est en retrait : agrégations Java au lieu de SQL GROUP BY, N+1 applicatifs sur `loadJournalEntryResponse` et `TaxService`, persistance 1-par-1 sans batching JDBC. Les manquants structurants pour la prod à 1000+ users sont nombreux : RLS non câblé Java, pagination absente sur audit-trail, pas de partitionnement temporel, pool Hikari sizing 50 users, @Async non borné, pas HTTP/2/gzip, RateLimitFilter non distribué. Effort estimé pour atteindre 9/10 (production-ready 1000+ users) : ~25 jours-homme sur les P0+P1.

Analyse conformité légale Haïti

L'analyse de la conformité légale haïtienne révèle un manque structurant : le projet a été développé sans prise en compte explicite du cadre juridique haïtien. Trois textes réglementaires sont particulièrement pertinents pour un SaaS comptable déployé en Haïti : le **Décret du 23 mai 2018 sur la protection des données à caractère personnel**, le **Code Fiscal Haïtien** (conservation des pièces comptables 10 ans, mentions légales factures art. 196), et le **Décret du 12 février 2002 sur la signature électronique**. Le score de 3/10 reflète une absence quasi totale de prise en compte de ces textes.

12.1 Décret du 23 mai 2018 — protection des données personnelles

Le Décret du 23 mai 2018 sur la protection des données à caractère personnel impose aux responsables de traitements plusieurs obligations : consentement explicite des personnes physiques, droit d'accès et de rectification, droit à l'effacement (« droit à l'oubli » sous réserve des obligations fiscales de conservation), notification de breach sous 72h à la Commission Sur la Protection des Données Personnelles (CSPDP), tenue d'un registre des traitements. La loi crée la CSPDP comme autorité de contrôle.

Aucune mention du Décret du 23 mai 2018 n'apparaît dans le code, le README, ou OPERATIONS.md. Le OPERATIONS.md §6 mentionne bien un conflit entre droit à l'oubli (RGPD) et obligation fiscale (10 ans rétention), mais fait référence au RGPD européen et non au décret haïtien. Aucun endpoint `/api/v1/privacy/consent`, `/api/v1/privacy/export`, `/api/v1/privacy/erase` n'existe — l'utilisateur ne peut pas exercer ses droits d'accès, rectification ou effacement.

Le `PiiMasker` masque les données personnelles dans les JSON d'audit (email, fullName, phone), ce qui est une bonne pratique, mais insuffisant pour la conformité. Il n'y a pas de procédure de notification de breach documentée (pas de runbook spécifique CSPDP). Pas de registre des traitements. La conservation par défaut des `refresh_token` (14 jours), `password_reset_token` (1 heure) et `audit_log` (illimitée) n'est pas documentée dans un cadre de conformité RGPD/décret haïtien.

12.2 Conservation des pièces comptables — Code Fiscal Haïtien

Le Code Fiscal Haïtien (et le Décret du 24 septembre 1985 sur la comptabilité) impose la conservation des pièces comptables pendant **10 ans**. Cette obligation s'applique aux factures, journaux comptables, bilans, déclarations fiscales, et tout document justifiant les écritures comptables. Pour un SaaS comptable hébergeant les données de plusieurs entreprises haïtiennes, cette obligation est doublement critique : (1) le SaaS doit garantir la disponibilité et l'intégrité des données sur 10 ans, (2) le SaaS doit empêcher toute altération rétroactive des pièces comptables.

Couverture partielle : le OPERATIONS.md §4.1 mentionne bien « 10 yearly (fiscal FR) » — mais c'est en référence à la LPF L102B française, pas au Code Fiscal haïtien. Le S3 Object Lock mode Compliance 10 ans est documenté dans `S3FileStorageAdapter` — c'est une bonne chose, mais insuffisant car il ne couvre que les fichiers stockés sur S3 (PDF, exports), pas les données en base.

Manquants critiques : la table `audit_log` n'est pas immutable (pas de trigger BEFORE UPDATE/DELETE, pas de REVOKE — voir section 8.3). Or, l'audit trail est précisément le mécanisme qui doit garantir l'intégrité forensique sur 10 ans. Un attaquant (ou un admin DB négligent) peut altérer ou supprimer l'audit. Pas de mention explicite du délai haïtien (10 ans selon Code Fiscal) dans la documentation applicative. Pas de WORM (Write Once Read Many) sur les tables critiques.

12.3 Signature électronique — Décret du 12 février 2002

Le Décret du 12 février 2002 sur la signature électronique en Haïti reconnaît la validité de la signature électronique (art. 5) à condition qu'elle soit créée avec un certificat qualifié délivré par une autorité de certification accréditée. Pour un SaaS comptable, cela concerne potentiellement : la signature des factures électroniques, la signature des déclarations fiscales DGI, la signature des bulletins de paie, la signature des états financiers. La DGI Haïti exige depuis l'arrêté du 4 octobre 2017 des NIF valides sur les factures, et certaines déclarations doivent être signées électroniquement.

Aucune mention de la loi haïtienne sur les signatures électroniques dans le code ou la documentation. Le `OPERATIONS.md` §4.4 mentionne « Signature : à ajouter — PAdES-B-LT (eIDAS qualifié) » — c'est le standard européen eIDAS, pas la loi haïtienne. Aucune signature XMLDSig/XAdES n'est implémentée dans le module invoicing. Aucun horodatage TSA (Time Stamp Authority) qualifié n'est intégré pour les PDF fiscaux. Aucune autorité de certification haïtienne n'est référencée.

12.4 Mentions légales factures — Code Fiscal art. 196

Le Code Fiscal Haïtien (art. 196) impose des mentions obligatoires sur les factures : NIF de l'émetteur, raison sociale, adresse, date de la facture, désignation des biens/services, montant HT, taux TVA/TCA applicable, montant des taxes, montant TTC. Toute facture non conforme expose l'émetteur à une amende de 5 000 à 50 000 HTG par facture.

Non-conformité majeure : les mentions légales des factures (migration V39 lignes 113-127) sont en dur et 100% françaises : « Mentions legales (CGI art. 289 + C. com. art. L441-10) », « Penalites de retard: 3x taux d'interet legal », « Indemnite forfaitaire pour frais de recouvrement: 40 EUR ». **Aucune mention du Code Fiscal haïtien**. Sanction : amende 5 000 à 50 000 HTG par facture non conforme — pour une entreprise émettant 100 factures par mois, le risque financier peut atteindre 5 millions de HTG par mois en cas de contrôle DGI.

12.5 Verdict conformité légale : 3 / 10 — Critique

Le projet n'est pas conforme au cadre juridique haïtien. Les trois textes structurants (Décret 23 mai 2018 sur les données personnelles, Code Fiscal Haïtien pour la conservation et les mentions légales, Décret 12 février 2002 sur la signature électronique) ne sont pas explicitement pris en compte. Cette non-conformité expose le SaaS à des risques juridiques et fiscaux majeurs en cas de déploiement en Haïti. Les actions prioritaires listées dans la section 14 (registre des traitements, endpoints privacy, audit_log immutable, signatures électroniques conformes) doivent être exécutées avant tout go-live.

Matrice de risques consolidée

Cette matrice consolide l'ensemble des risques identifiés par les 5 expertises mobilisées, classés par priorité P0 (critique, bloquant production), P1 (important, à corriger dans les 60 jours) et P2 (amélioration continue). Chaque risque est évalué sur sa probabilité d'occurrence et son impact, avec une action de mitigation associée. Cette matrice constitue la base du plan d'action 30/60/90 jours présenté dans la section suivante.

13.1 Risques P0 — Bloquants absolus pour la production

ID	Risque	Catégorie	Proba.	Impact
R-01	Bypass authentification MFA (parseClaims sans vérif signature)	Sécurité	Élevée	Critique
R-02	APP_MFA_ENCRYPTION_KEY sans fail-fast (clé dev en prod)	Sécurité	Moyenne	Critique
R-03	RLS PostgreSQL inactif (GUC app.current_tenant jamais SET)	Sécurité	Élevée	Critique
R-04	Audit_log non immutable (pas de trigger/REVOKE)	Conformité	Élevée	Critique
R-05	IS calculé au taux français (25% au lieu de 30% HT)	Fiscal	Élevée	Critique
R-06	Calendrier déclaratif 100% français (pas de DCR/DCRf HT)	Fiscal	Certaine	Critique
R-07	TCA haïtienne absente (Code Fiscal art. 196)	Fiscal	Certaine	Élevé
R-08	Mentions légales factures 100% CGI (amende 5K-50K HTG/facture)	Légal	Certaine	Élevé
R-09	GET /audit-trail non paginé → OOM certain	Perf	Certaine	Critique
R-10	N+1 TaxService.getDeclaration (1000 factures = 1003 requêtes)	Perf	Élevée	Élevé
R-11	getBalanceSheet/getCashFlow en Java au lieu de SQL GROUP BY	Perf	Élevée	Élevé
R-12	CI/CD sans scans SAST/SCA/image (CVEs non détectées)	DevOps	Élevée	Élevé
R-13	Bouncy Castle 1.78.1 avec 2 CVEs publics connus	Sécurité	Certaine	Élevé
R-14	RateLimitFilter in-memory non distribué (multi-instances)	Sécurité	Élevée	Élevé
R-15	ArchUnit Rule 28 probablement cassée (silence alarmant)	Qualité	Moyenne	Moyen

13.2 Risques P1 — À corriger dans les 60 jours

ID	Risque	Catégorie	Proba.	Impact
R-16	27 modules à 0% de tests unitaires (refactoring risqué)	Qualité	Certaine	Élevé
R-17	Bypass MFA : /auth/login/mfa non rate-limité	Sécurité	Élevée	Élevé
R-18	Java 17 au lieu de 21 (virtual threads no-op)	Perf	Certaine	Moyen
R-19	Pool HikariCP=30 sizing 50 users (pas 1000+)	Perf	Certaine	Élevé
R-20	Paie : constantes FR hardcodées (173.33h, HS 25/50%)	Fiscal	Certaine	Élevé
R-21	Factur-X sans NIF HT (arrêté DGI 4 oct 2017)	Fiscal	Certaine	Élevé
R-22	PDF/A-3 Factur-X non implémenté (UnsupportedOperationException)	Fiscal	Certaine	Moyen
R-23	Crédit TVA non persisté (TODO TaxService:247)	Fiscal	Moyenne	Moyen
R-24	Aucune TCA modélisée (Code Fiscal art. 196)	Fiscal	Certaine	Élevé

ID	Risque	Catégorie	Proba.	Impact
R-25	CNSS/OFATMA/AST non seedés (régime HT_GENERAL vide)	Fiscal	Certaine	Élevé
R-26	CORS * par défaut avec allowCredentials=true	Sécurité	Moyenne	Élevé
R-27	Pas de cache distribué Redis L2 (staleness multi-instances)	Perf	Certaine	Moyen
R-28	Imports bancaires save 1-par-1 sans batching JDBC	Perf	Certaine	Moyen
R-29	@Async sans ThreadPoolTaskExecutor (explosion threads)	Perf	Moyenne	Moyen
R-30	Pas de NetworkPolicy/PDB/ServiceMonitor dans Helm	DevOps	Moyenne	Moyen
R-31	Pas de dashboards Grafana ni règles Alertmanager	DevOps	Certaine	Moyen
R-32	Décret 23 mai 2018 absent (registre, endpoints privacy)	Légal	Certaine	Élevé
R-33	Signature électronique non conforme Décret 2002	Légal	Moyenne	Moyen

13.3 Risques P2 — Amélioration continue post-production

ID	Risque	Catégorie	Proba.	Impact
R-34	Anemic domain model (invariants éparpillés)	Architecture	Certaine	Moyen
R-35	God classes (FixedAssetsService 1366 LOC, etc.)	Qualité	Certaine	Moyen
R-36	38 TODO/FIXME dans le code (backlog implicite)	Qualité	Certaine	Faible
R-37	Pas de partitionnement audit_log (100M+ lignes / 5 ans)	Perf	Certaine	Moyen
R-38	Pas de tests de mutation PIT	Qualité	Certaine	Faible
R-39	Pas de chaos engineering	DevOps	Faible	Moyen
R-40	Pas de tests de charge sur endpoints critiques (audit-trail)	Perf	Certaine	Moyen
R-41	Pas d'HIBP sur password validator	Sécurité	Moyenne	Faible
R-42	JWKS sans rotation de clés	Sécurité	Moyenne	Moyen
R-43	Pas de Spring Modulith (vérification runtime boundaries)	Architecture	Faible	Faible
R-44	Pas de diagrammes PlantUML/Mermaid dans docs	Documentation	Certaine	Faible
R-45	PCN_HAITI traité comme SYSCOHADA renommé	Fiscal	Certaine	Moyen

13.4 Synthèse — 45 risques identifiés

L'audit a identifié **45 risques au total** : 15 P0 (bloquants absolus pour la production), 18 P1 (à corriger dans les 60 jours) et 12 P2 (amélioration continue post-production). La répartition par catégorie montre une concentration particulière sur les dimensions Sécurité (10 risques), Fiscalité Haïti (10 risques) et Performance (9 risques) — ces trois dimensions constituent le cœur du travail de remédiation.

Lecture stratégique de la matrice

Aucun risque P0 n'est tolérable pour un go-live — chacun expose le projet à un risque juridique, fiscal ou de sécurité immédiat. Les 15 P0 doivent être intégralement corrigés avant le lancement.

Les 18 P1 doivent être traités dans les 60 jours suivant le go-live pour atteindre un niveau de maturité production-ready acceptable. Sans cette remédiation P1, le SaaS fonctionne mais reste fragile (coups de semonce probables).

Les 12 P2 relèvent de l'amélioration continue et peuvent être traités dans le cadre de la roadmap produit post-lancement.

Plan d'action 30/60/90 jours

Ce plan d'action traduit la matrice de risques en roadmap concrète sur 3 mois, avec une estimation réaliste de la charge de travail. L'objectif est d'atteindre un état GO conditionnel à J+30 (correction des P0 bloquants), un état production-ready à J+60 (correction des P1 structurants), et un état mature à J+90 (correction des P2 prioritaires et ouverture à un beta privé). Le budget total estimé est d'environ 110 jours-homme, soit 6-8 semaines avec 2 développeurs expérimentés.

14.1 J+30 — Sprint P0 (8 actions critiques, ~25 jh)

Le sprint J+30 concentre les 8 actions bloquantes pour la production. Aucune d'entre elles ne relève de l'amélioration continue : chacune expose le projet à un risque juridique, fiscal ou de sécurité immédiat. La charge estimée de 25 jh correspond à 5 semaines-homme, réalisable en 3 semaines calendaires avec 2 développeurs.

#	Action	Fichier(s)	Effort
1	Corriger bypass MFA (valider signature via JwtDecoder, pas parseClaims)	AuthController.loginMfa, JwtService.parseClaims	2 jh
2	Fail-fast APP_MFA_ENCRYPTION_KEY (@PostConstruct dans MfaService)	auth/service/MfaService.java	0.5 jh
3	Câbler RLS PostgreSQL (SET LOCAL app.current_tenant au début de chaque transaction)	core/tenant/TenantContextFilter + nouveau ConnectionCustomizer	2 jh
4	Audit_log immutable (trigger BEFORE UPDATE/DELETE + REVOKE)	migration V53 + revoke SQL	1 jh
5	Paginer GET /audit-trail + GET /journal-entries legacy	AuditTrailController, AccountingEngineController	1 jh
6	Réécrire getBalanceSheet/getIncomeStatement/getCashFlow/getTrialBalance en SQL GROUP BY	FinancialStatementsService, AccountingEngineService	5 jh
7	Réécrire TaxService.getDeclaration en 1 requête SQL GROUP BY	tax/service/TaxService.java	1 jh
8	Bump Bouncy Castle 1.78.1 → 1.79+ + aligner Dockerfile et build.gradle.kts sur Java 21	libs.versions.toml, Dockerfile, build.gradle.kts	1 jh
9	Vérifier et corriger ArchUnit Rule 28 (notifications vs modules business)	ArchUnitTest.java, build.gradle.kts notifications	1 jh
10	Réparer MfaController.java:95 (récupérer le vrai email)	auth/controller/MfaController.java	0.5 jh
11	Ajouter ThreadPoolTaskExecutor borné pour @Async	nouveau AsyncConfig.java	0.5 jh
12	Configurer importStatement saveAll + hibernate.jdbc.batch_size=50	BankReconciliationService, application.yml	1 jh
13	defaultCorporateTaxRule(country) — IS=30% pour HT, 25% pour FR	tax/service/TaxService.java	1 jh
14	Implémenter HaitianTaxDeclarationSchedule (TVA/TCA/RS/acompte IS mensuel + DCR annuelle)	nouveau HaitianTaxDeclarationSchedule.java	3 jh
15	Seeder TaxRule/ContributionRule globales HT (TVA 10%, TCA, CNSS, OFATMA, AST)	migration V54 + seeder	3 jh

Livrable J+30 — État GO conditionnel

À l'issue du sprint J+30, le projet aura corrigé les 15 actions P0 critiques. Les 3 vulnérabilités de sécurité critiques seront résolues (bypass MFA, clé MFA, RLS), l'audit_log sera immuable (conformité Code Fiscal Haïtien), le calendrier déclaratif DGI sera implémenté, et les goulots d'étranglement performance majeurs seront éliminés. **Le projet sera alors en état GO CONDITIONNEL** : déployable en beta privé avec 3-5 PME haïtiennes pilotes, sous surveillance renforcée.

14.2 J+60 — Sprint P1 (12 actions structurantes, ~50 jh)

Le sprint J+60 traite les 18 risques P1. L'objectif est d'atteindre un état production-ready robuste, capable de supporter 1000+ utilisateurs concurrents en multi-tenant avec une conformité fiscale haïtienne acceptable. La charge estimée de 50 jh correspond à 10 semaines-homme, réalisable en 5 semaines calendaires avec 2 développeurs.

#	Action	Effort
16	Migration Java 17 → 21 + activation virtual threads nativement	1 jh
17	Tests unitaires sur modules critiques (tax, payroll, invoicing, fixed-assets, accounting-engine)	15 jh
18	Activer jacocoTestCoverageVerification (failOnViolation=true) + SpotBugs/PMD fail	2 jh
19	DgiHaitiExportService (DCR, DCRf, DCRG, DCLS au format DGI)	5 jh
20	Templates facture/avoir/bulletin de paie HT (Code Fiscal art. 196 + Code du Travail)	5 jh
21	FacturXExporter.writeTradeParty + NIF HT + openpdf pour PDF/A-3	2 jh
22	Étendre V50 pour seed comptes 442/446/447/4438 aussi pour PCN_HAITI	1 jh
23	Rendre MONTHLY_LEGAL_HOURS configurable + overtimeHours100 + cnssNumber/ofatmaSectorCode	3 jh
24	Migrer Bucket4jRateLimitStore vers Redis (bucket4j-redis + Lettuce)	2 jh
25	Cache distribué Redis L2 pour accounts/journals/taxRules	5 jh
26	CD pipeline : push image ghcr.io + helm upgrade --atomic + helm test	3 jh
27	Pipeline CI sécurité : Trivy + Dependabot + CodeQL + Cosign + Syft SBOM	2 jh
28	Dashboards Grafana + règles Alertmanager + SLO formels (99.9% dispo, P99<1s)	3 jh
29	Image distroless (cgr.dev/chainguard/jdk) + NetworkPolicy + PDB + ServiceMonitor	2 jh
30	Activer @TenantId Hibernate 6 (auto-filtre SELECT cross-tenant)	5 jh
31	Externaliser taux IS dans table corporate_tax_rate + calcul Charasse + crédit TVA	3 jh
32	Conformité Décret 23 mai 2018 : registre traitements + endpoints privacy + breach 72h	5 jh
33	Réécrire loadJournalEntryResponse en batch IN + getStatus en SQL COUNT/SUM	2 jh

Livrable J+60 — État production-ready robuste

À l'issue du sprint J+60, le projet aura corrigé les 18 risques P1 structurants. Le SaaS sera capable de supporter 1000+ utilisateurs concurrents en multi-tenant avec cache distribué, rate limiting distribué, et observabilité complète (dashboards Grafana + Alertmanager + SLO). La conformité fiscale haïtienne sera acceptable (export DGI, mentions légales, Factur-X avec NIF HT). **Le projet sera alors en état production-ready robuste**, déployable à plus large échelle.

14.3 J+90 — Sprint P2 (10 actions d'amélioration, ~35 jh)

Le sprint J+90 traite les 12 risques P2 prioritaires. L'objectif est d'atteindre un état mature avec amélioration continue outillée (tests de mutation, chaos engineering, signatures électroniques conformes). Cette phase peut être conduite en parallèle du beta privé lancé en J+30 et étendu progressivement.

#	Action	Effort
34	Introduire des aggregates riches (JournalEntry.addLine, SalesInvoice.issue, PayrollRun.approve)	10 jh
35	Intégration HIBP (k-anonymity) pour le password validator	1 jh
36	Signature électronique conforme Décret 2002 (autorité haïtienne ou eIDAS qualifié)	5 jh
37	Partitionner audit_log par mois via pg_partman + rétention 7 ans	3 jh
38	Chaos engineering (chaos-mesh) : injection latence DB, kill pod, network partition	3 jh
39	PIT mutation testing sur core/accounting-engine/payroll/tax	3 jh
40	Scinder :reporting en :reporting-core + :reporting-exports	2 jh
41	Keyset pagination sur journal-entries/search	2 jh
42	Créer PcnHaitiAccountTemplate + branche inferReportingClass spécifique	3 jh
43	Test de restauration cross-region trimestriel	1 jh
44	Spring Modulith (vérification runtime boundaries) + ApplicationModules.verify()	1 jh
45	Compléter la documentation : diagrammes PlantUML + glossaire DGI/DCL/CNSS/OFATMA	1 jh

14.4 Budget et staffing

Le budget total estimé pour atteindre l'état production-ready robuste (J+60) est de **~75 jours-homme** (25 jh P0 + 50 jh P1). Avec 2 développeurs expérimentés Java/Spring + 1 expert-comptable haïtien DGI en relecture (à mi-temps), cela représente **6-8 semaines calendaires**. Le sprint J+90 P2 (~35 jh supplémentaires) peut être conduit en parallèle du beta privé sans impact sur le go-live.

Recommandation staffing : (1) 1 développeur backend senior Java/Spring à temps plein sur les 3 sprints, (2) 1 développeur backend confirmé à temps plein sur les sprints J+30 et J+60, (3) 1 expert-comptable haïtien DGI en consultancy à mi-temps (relecture des spécifications fiscales, validation des barèmes CNSS/OFATMA/AST, conformité DCR/DCRf), (4) 1 devops/SRE à temps partiel (CD pipeline, dashboards Grafana, scans sécurité).

14.5 Recommandation de lancement progressif

Phase 1 — Beta privé (J+30 à J+60) : déploiement en beta privé avec 3-5 PME haïtiennes pilotes sélectionnées (idéalement 1 commerce retail, 1 cabinet de services, 1 ONG, 1 société en zone franche). Surveillance renforcée (alerting P1, runbook dédié, on-call). Objectif : valider la conformité fiscale en conditions réelles, détecter les bugs edge-case, recueillir le feedback utilisateur.

Phase 2 — Beta élargie (J+60 à J+90) : extension à 20-30 entreprises haïtiennes avec onboarding assisté. Objectif : valider la scalabilité (100+ tenants, 1000+ utilisateurs), éprouver les opérations (backup/restore en réel, disaster recovery test), industrialiser le support client.

Phase 3 — GA (General Availability) à J+90 : ouverture à l'inscription publique avec pricing SaaS, documentation utilisateur, support client formalisé. À ce stade, le projet aura démontré sa conformité

fiscale, sa robustesse opérationnelle et son adéquation au marché haïtien. La matrice de risques résiduelle sera dominée par les P2 qui relèvent de l'amélioration continue.

Verdict Go/No-Go final & conditions

VERDICT FINAL

NO-GO CONDITIONNEL

Au terme de cet audit multi-experts approfondi portant sur 30 modules Gradle, 464 fichiers Java, 63 migrations Flyway et 310 tests, le verdict final est **NO-GO CONDITIONNEL**. Le backend JOAccountant v5.2 présente une **architecture solide et bien documentée** qui démontre une réelle maîtrise du DDD stratégique et de l'ingénierie Spring Boot, mais il **n'est pas prêt pour une mise en production sur le marché haïtien en l'état**.

Trois catégories de blocages critiques ont été identifiées, totalisant 15 risques P0 bloquants : (1) **3 vulnérabilités de sécurité critiques** avec bypass MFA, clé MFA sans fail-fast, RLS PostgreSQL inactif ; (2) **une inadaptation fiscale majeure** au contexte haïtien avec calendrier 100% français, TCA absente, taux IS hardcoded à 25% au lieu de 30%, mentions légales factures 100% CGI ; (3) **des lacunes ops structurantes** avec audit_log non immutable, pipeline CI/CD sans scans sécurité, pagination manquante sur /audit-trail (OOM certain).

15.1 Les 3 conditions absolues pour atteindre GO

Le verdict basculera de NO-GO conditionnel à GO lorsque les 3 conditions absolues suivantes seront intégralement satisfaites. Aucune d'entre elles n'est négociable — chacune expose le projet à un risque juridique, fiscal ou de sécurité immédiat en cas de non-respect.

Condition 1 — Corriger les 3 vulnérabilités critiques de sécurité

Ces 3 vulnérabilités constituent des bloquants absolus car elles exposent directement les utilisateurs du SaaS à des attaques (prise de compte, déchiffrement de secrets, fuite cross-tenant). Aucune ne relève de l'amélioration continue.

- **Corriger le bypass MFA** : valider la signature du mfaChallengeToken via JwtDecoder (pas parseClaims), passer le token en header/body (pas query param), étendre RateLimitFilter à /auth/login/mfa. Effort : 2 jh.
- **Fail-fast APP_MFA_ENCRYPTION_KEY** : ajouter @PostConstruct dans MfaService qui refuse le pattern dev-only-mfa-key en profil prod. Effort : 0.5 jh.
- **Câbler RLS PostgreSQL** : implémenter un ConnectionCustomizer Hibernate qui exécute SET LOCAL app.current_tenant = ? au début de chaque transaction. BYPASSRLS pour le user Flyway. Effort : 2 jh.

Condition 2 — Implémenter le socle fiscal Haïti minimum

Sans ces éléments, le SaaS ne peut pas être utilisé légalement par une entreprise haïtienne pour sa comptabilité fiscale. Chaque écart expose l'utilisateur à des redressements DGI et à des pénalités substantielles.

- **Calendrier déclaratif DGI** : implémenter HaitianTaxDeclarationSchedule avec TVA/TCA/RS mensuelles (15 M+1), acompte IS 1% mensuel, DCR/DCRf annuelles (31 mars N+1). Effort : 3 jh.
- **TCA modélisée** : ajouter TaxRule.taxType enum (VAT, TCA, TURNOVER_TAX) + seeds TaxRule globales HT (TVA_HT_10, TCA_HT_2_BANK, TCA_HT_5_TELECOM, TCA_HT_10_SERVICES). Effort :

1 jh.

- **Taux IS haïtien par défaut** : remplacer defaultFrenchCorporateTaxRule par defaultCorporateTaxRule(country) qui renvoie IS=30% pour HT, 25% pour FR. Effort : 1 jh.
- **Mentions légales factures Code Fiscal art. 196** : créer des templates facture/avoir distincts par country (template HT avec NIF émetteur, raison sociale, adresse, taux TVA/TCA, montant HT/TTC). Effort : 3 jh.
- **Seeds CNSS/OFATMA/AST** : peupler ContributionRule HT_GENERAL (CNSS 6%+6%, OFATMA Santé 3%+1%, OFATMA Accidents 2-3%, AST barème progressif 2024). Effort : 3 jh.

Condition 3 — Sécuriser l'ops minimum

Ces éléments garantissent que le SaaS fonctionnera de façon fiable en production et que l'audit forensique sera conforme aux obligations légales haïtiennes de conservation sur 10 ans.

- **Audit_log immutable** : trigger BEFORE UPDATE/DELETE + REVOKE UPDATE, DELETE sur le rôle applicatif. Conformité Code Fiscal Haïtien 10 ans. Effort : 1 jh.
- **Pagination /audit-trail** : paginer avec filtres ?page=&size;=&entityType;=&actorUserId;=&from;=&to;= pour éviter l'OOM certain sur 100M+ lignes. Effort : 1 jh.
- **CD pipeline avec SAST/SCA** : ajouter jobs Trivy (image + filesystem), Dependabot, CodeQL, Cosign, Syft SBOM. Faire fail le build sur CRITICAL. Effort : 2 jh.

15.2 Timeline réaliste pour atteindre GO

Avec 2 développeurs backend expérimentés Java/Spring + 1 expert-comptable haïtien DGI en relecture à mi-temps, les 3 conditions absolues (10 actions critiques, ~17 jh) peuvent être satisfaites en **3 semaines calendaires**. Le sprint J+30 du plan d'action (15 actions P0, ~25 jh) peut être complété en **5 semaines calendaires**. La timeline réaliste pour atteindre un GO conditionnel solide (J+30 complet) est donc de **6-8 semaines**.

Cette timeline suppose : (1) disponibilité immédiate de l'équipe, (2) accès à un expert-comptable haïtien pour valider les barèmes CNSS/OFATMA/AST et les formats DCR/DCRf, (3) environnement de staging avec PostgreSQL 14+ et S3/MinIO pour tester le câblage RLS, (4) accès à un cluster Kubernetes de test pour valider le Helm chart durci.

15.3 Recommandation de lancement progressif

Plutôt que de viser un go-live direct en production publique, nous recommandons un **lancement progressif en 3 phases** : (1) **Beta privé (J+30 à J+60)** avec 3-5 PME haïtiennes pilotes sélectionnées (1 commerce retail, 1 cabinet de services, 1 ONG, 1 société en zone franche) sous surveillance renforcée et on-call dédié ; (2) **Beta élargie (J+60 à J+90)** avec 20-30 entreprises haïtiennes et onboarding assisté pour valider la scalabilité et éprouver les opérations en réel ; (3) **GA (General Availability) à J+90** avec ouverture à l'inscription publique, pricing SaaS, documentation utilisateur et support client formalisé.

Cette approche progressive permet de : (a) valider la conformité fiscale en conditions réelles avant l'ouverture publique, (b) détecter les bugs edge-case sur un périmètre maîtrisé, (c) industrialiser le support client avant le scale, (d) construire une base d'utilisateurs référents qui pourront témoigner lors du lancement public. Le coût d'une telle approche est minime (3 mois de beta) comparé au risque d'un lancement public précipité qui exposerait le SaaS à des redressements DGI en cascade.

15.4 Forces à conserver et opportunités

Au-delà des corrections critiques, l'audit a identifié **5 forces majeures à conserver absolument** : (1) l'architecture DDD stratégique exemplaire (26 bounded contexts = 26 modules Gradle), (2) la défense multi-tenant à 3 niveaux (filtre HTTP + listener JPA + RLS PostgreSQL), (3) l'observabilité production-grade (Prometheus + OpenTelemetry + logs JSON + MDC), (4) le backup PITR testé mensuellement (rare et excellent), (5) la documentation exceptionnelle (720 lignes README + 28 README par module + OPERATIONS.md + 197 @Operation Swagger).

Opportunités stratégiques pour le marché haïtien : (1) premier SaaS comptable haïtien nativement multi-référentiel (PCN_HAITI + IFRS + SYSCOHADA), (2) support natif multi-devises HTG/USD (critique pour les entreprises dollarisées), (3) module funds-grants pertinent pour les nombreuses ONG haïtiennes (CRS, World Vision, Care), (4) Factur-X EN 16931 comme base pour une future facturation électronique conforme DGI, (5) application mobile livrée (APK 21.7 MB, 16 feature modules) — différentiateur pour le marché mobile-first haïtien.

15.5 Mot de la fin

Le projet JOAccountant Backend v5.2 est **engagé sur la bonne voie architecturale** mais doit **corriger sa trajectoire fiscale** pour le marché cible haïtien. Les fondations techniques permettent une remontée rapide vers l'état production-ready en 6-8 semaines, à condition de mobiliser les ressources nécessaires (2 développeurs + 1 expert-comptable DGI). Le potentiel est réel : il n'existe à ce jour aucun SaaS comptable haïtien nativement multi-référentiel avec cette maturité technique. Avec les corrections P0/P1 et un lancement progressif en beta privé, JOAccountant peut devenir une référence du marché comptable haïtien sur 12-18 mois.

Annexes — Tableaux détaillés

Annexe A — Liste exhaustive des 38 TODO/FIXME

Les 38 commentaires TODO/FIXME identifiés dans le code source sont répartis comme suit : tax (17), auth (6), invoicing (5), app (4), notifications (2), autres modules (4). 32 sont étiquetés "TODO v4.8" / "TODO v4.9" (backlog explicite — bonne pratique), 6 sont des TODO génériques sans version cible. Les 3 plus critiques sont mentionnés en section 10.4 : TaxService.java:247 (crédit TVA), TaxService.java:315 (calcul Charasse), MfaController.java:95 (email MFA fictif). La liste complète est disponible dans le dépôt via `grep -rn "TODO\|FIXME\|HACK" --include="*.java"`.

Annexe B — Vulnérabilités CVE potentielles sur les dépendances

Sans scan SCA automatisé (Trivy, Snyk, OWASP Dependency-Check — voir section 9.1), l'audit manuel a identifié les CVEs suivants sur les versions explicitement déclarées dans `gradle/libs.versions.toml` :

Dépendance	Version actuelle	CVE	Sévérité	Correctif
Bouncy Castle	1.78.1	CVE-2024-29857 (DoS EACertificate)	Élevée	1.79+
Bouncy Castle	1.78.1	CVE-2024-30171 (timing attack RSA TLS)	Moyenne	1.79+
Caffeine	3.1.8	Aucun CVE mais versions 3.2.x recommandées	Faible	3.2.x
ShedLock	5.10.0	Aucun CVE mais 6.x est la version courante	Faible	6.x
Java	17	Pas de CVE mais Java 21 LTS recommandé (virtual threads)	Faible	21

Recommandation : intégrer un scan automatique Trivy ou OWASP Dependency-Check dans le pipeline CI/CD (cf. action #27 du plan 90 jours) pour détecter en continu les CVEs sur les dépendances transitives non trackées dans le catalog de versions.

Annexe C — Glossaire des acronymes

Ce glossaire regroupe les acronymes techniques et fiscaux utilisés dans ce rapport, principalement liés au contexte fiscal haïtien et au vocabulaire comptable francophone.

Acronyme	Signification	Contexte
AST	Ajustement Social Temporaire	Cotisation sociale haïtienne, barème progressif par tranches mensuelles (Décret 2017 modifié)
CGI	Code Général des Impôts (français)	Référentiel fiscal français, utilisé par erreur dans le code
CNSS	Caisse Nationale de Sécurité Sociale (Haïti)	Cotisation 6% employeur + 6% salarié, plafonnée à 6 PMT
CSPDP	Commission Sur la Protection des Données Personnelles	Autorité de contrôle du Décret 23 mai 2018 Haïti
DGI	Direction Générale des Impôts (Haïti)	Administration fiscale haïtienne
DCR	Déclaration Comptable et Fiscale Résumée	Déclaration annuelle DGI, échéance 31 mars N+1
DCRf	DCR feuillet 1 (résumé)	Annexe de la DCR

Acronyme	Signification	Contexte
DCRG	DCR Grand livre	Annexe de la DCR
DCLS	État financier (bilan, CR, notes)	Annexe de la DCR
DDD	Domain-Driven Design	Méthodologie de conception logicielle
OFATMA	Office d'Assurance Accidents du Travail, Maladie et Maternité	Cotisation santé 3% employeur + 1% salarié, accidents 2-3%
PCN	Plan Comptable National (Haïti)	Référentiel comptable haïtien, 8 classes
PCG	Plan Comptable Général (France)	Référentiel comptable français, 7 classes
PMT	Plafond Mensuel des Traitements	Plafond CNSS Haïti $\approx 6 \times \text{SMG} \approx 150\,000$ HTG/mois
RS	Retenue à la Source	Code Fiscal Haïti art. 156 — 2% sur prestations locales
RLS	Row Level Security	Mécanisme PostgreSQL d'isolation des lignes par tenant
SMG	Salaire Minimum Général	SMIC haïtien, base de calcul PMT
SYSCOHADA	Système Comptable OHADA	Référentiel comptable africain francophone, 8 classes + TAFIRE
TAFIRE	Tableau d'Analyse Financière des Résultats et Emplois	État financier SYSCOHADA équivalent cash flow
TCA	Taxe sur le Chiffre d'Affaires	Code Fiscal Haïti art. 196 — 2% banques, 5% télécoms, 10% services
TVA	Taxe sur la Valeur Ajoutée	Haïti 10% (loi 24 juillet 2002), sur débits
UE	Union Européenne	Contexte Factur-X, reverse charge intra-UE

Annexe D — Références réglementaires

Code Fiscal Haïtien : articles 4 (IS 30%), 5 (acompte IS 1% sur encaissements), 156 (retenues à la source 2%), 191 (TVA sur débits), 196 (mentions obligatoires factures), 197 (TCA banques 2%), 240 (pénalités 10-50%). **Code du Travail Haïtien** : articles 4 (apprentissage), 32 (durée légale 48h/sem), 36 (heures sup 50%/100%), 153 (13 $\frac{1}{2}$ mois), 156 (congés 15 jours ouvrables). **Décret 23 mai 2018** sur la protection des données personnelles. **Décret 12 février 2002** sur la signature électronique. **Arrêté DGI 4 octobre 2017** sur la facturation électronique (NIF obligatoire).

OWASP Top 10 2021 : référence pour l'audit sécurité applicative. **NIST 800-63B** : référence pour l'authentification (AAL2 pour MFA). **ISO 27001** : référence pour la gestion de la sécurité de l'information. **RFC 7807** : Problem Details for HTTP APIs. **RFC 6238** : TOTP. **RFC 7517** : JWKS. **EN 16931** : Factur-X. **IAS 7 / IAS 16 / IAS 36 / IAS 21** : normes IFRS pour cash flow, immobilisations, dépréciation, devises.